ĐẠI HỌC BÁCH KHOA HÀ NỘI

Khoa Toán - Tin

——————— o0o ———————





BÁO CÁO CUỐI KÌ

KỸ THUẬT LẬP TRÌNH - MI3310

HỆ THỐNG QUẢN LÝ THƯ VIỆN

Giảng viên hướng dẫn: TS. Vũ Thành Nam
Mã lớp: 169312
Nhóm : G1

Thành viên:	Hoàng Thủy Phương	202419088
	Cao Đỗ Hoàng Sơn	202419090
	Hoàng Minh Sơn	202419092

HÀ NỘI, 2026

LỜI MỞ ĐẦU

Trong kỷ nguyên số, thư viện đại học không chỉ lưu trữ tài liệu mà còn là trung tâm phân phối tri thức. Tuy nhiên, khối lượng tài liệu khổng lồ cùng lượng lớn độc giả tạo ra áp lực lớn lên hệ thống vận hành. Việc quản lý thủ công bộc lộ nhiều yếu điểm: dễ sai sót trong thống kê, thất thoát tài liệu và lãng phí thời gian. Một hệ thống quản lý số hóa toàn diện là yêu cầu cấp thiết để giải quyết các nhu cầu:

Về phía quản trị (Admin/Thủ thư): Cần khả năng biên mục (thêm, sửa, xóa) hàng ngàn đầu sách với tốc độ truy xuất tức thời; theo dõi chặt chẽ dòng chảy mượn/trả tài liệu; tự động tính toán phí phạt quá hạn và trích xuất các báo cáo thống kê (sách quá hạn, sách mượn nhiều nhất) một cách nhanh chóng.

Về phía người dùng (Sinh viên/Giảng viên): Cần một cổng thông tin tra cứu tài liệu thuận tiện theo nhiều tiêu chí (tên sách, tác giả, thể loại), đồng thời có thể chủ động theo dõi lịch sử giao dịch và thời hạn trả sách của cá nhân.

Hiện nay, các phần mềm thương mại (ERP/Koha) thường quá cồng kềnh, chi phí cao và thừa tính năng đối với quy mô vừa và nhỏ. Ngược lại, giải pháp dùng Spreadsheet (Excel) lại thiếu tính toàn vẹn dữ liệu, truy xuất chậm khi dữ liệu lớn và không hỗ trợ phân quyền bảo mật triệt để.

Nhận thấy những hạn chế đó, nhóm quyết định phát triển **"Hệ thống Quản lý Thư viện Console"** bằng ngôn ngữ Python. Tránh phụ thuộc vào hệ quản trị CSDL nặng nề, hệ thống được tối ưu từ tầng thấp nhất với các đặc điểm nổi bật:

Cấu trúc dữ liệu chuyên biệt: Áp dụng Bảng băm (Hash Map) để tra cứu sách với độ phức tạp $\mathcal{O}(1)$; Danh sách liên kết kép (Doubly Linked List) lưu vết lịch sử giao dịch; và Hàng đợi ưu tiên (Max-Heap) để thống kê Top sách thịnh hành.

Kiến trúc phân tầng (Layered Architecture): Tách biệt rõ ràng các tầng Storage (Tệp CSV), Structure (RAM), Logic và Interface, giúp mã nguồn dễ bảo trì và mở rộng.

Giao diện tối giản, hiệu năng cao: Giao diện Console (CLI) loại bỏ độ trễ đồ họa, kết hợp thuật toán Quick Sort và phân trang tự động giúp hiển thị mượt mà hàng ngàn bản ghi.

Đề tài không chỉ giải quyết trọn vẹn nghiệp vụ thư viện thực tế mà còn minh chứng cho việc ứng dụng hiệu quả các lý thuyết về Cấu trúc dữ liệu và Giải thuật vào tối ưu hóa phần mềm.

Mục lục

I Mô tả thiết kế **5**

1 Giới thiệu chung . . . 5
2 Mô hình hóa bài toán . . . 5
3 Cấu trúc dự án . . . 6
4 Mô tả sơ lược về chương trình . . . 6
5 Mô tả chi tiết chương trình . . . 6

II Cấu trúc dữ liệu tự cài đặt **10**

1 Mảng động tự cài đặt — CustomList . . . 10
2 Danh sách liên kết kép — TransactionList . . . 10
3 Bảng băm tự cài đặt — BookHashMap . . . 11
4 Hàng đợi có ưu tiên — PriorityQueue (Max-Heap nhị phân) . . . 12
5 Hàng đợi FIFO — WaitingQueue (Danh sách liên kết đơn) . . . 12
6 Mảng quản lý người dùng — UserArray . . . 13
7 Về các phương thức đặc biệt (Dunder methods) . . . 13

III Cấu trúc đối tượng **14**

1 Cấu trúc "Book" . . . 14
2 Cấu trúc "User" . . . 17
3 Cấu trúc "Loan" . . . 21
4 Cấu trúc "BorrowRequest" . . . 25

IV Logic nghiệp vụ (Thuật toán xử lý) **28**

1 Tìm kiếm sách — `search.py` . . . 28
2 Sắp xếp danh mục — Quick Sort tự cài (`sort.py`) . . . 28
3 Điều phối mượn/trả sách — `loan_manager.py` . . . 29
4 Báo cáo thống kê — `report.py` . . . 31

V Xây dựng giao diện **32**

1 Giao diện chính của chương trình . . . 32
2 Giao diện và thao tác với Sách . . . 34
3 Giao diện và thao tác với Bạn đọc . . . 35
4 Giao diện và thao tác với Mượn/Trả sách . . . 36
5 Giao diện Báo cáo Thống kê (Admin) . . . 37
6 Giao diện Độc giả (User Portal) . . . 38

VI Kiểm thử **40**

1 Phương pháp và công cụ kiểm thử 40
2 Cơ chế tổ chức và đánh giá một trường hợp kiểm thử 40
3 Nguyên tắc lựa chọn dữ liệu kiểm thử 41
4 Kết quả kiểm thử 42

VII Kĩ thuật lập trình áp dụng 43
1 Kĩ thuật thiết kế Top-Down 43
2 Kĩ thuật module hoá 43
3 Cấu trúc dữ liệu và giải thuật 44
4 Xử lý lỗi và ghi log 45
5 Đặt tên rõ ràng và nhất quán 45
6 Chú thích 45
7 Lập trình phòng ngừa 46
8 Đánh giá tổng kết 47

VIII Kết luận 48

IX Mã nguồn 49
1 Cấu trúc mã nguồn 49
2 Mô tả file dữ liệu 50
3 Chi tiết file main.py 52
4 Phụ lục link code 55

I Mô tả thiết kế

1 Giới thiệu chung

Trong thời đại số hóa, việc quản lý thư viện theo phương thức truyền thống qua ghi chép sổ sách bộc lộ nhiều bất cập về tốc độ tra cứu, tính toàn vẹn dữ liệu và khó khăn trong việc kiểm soát luân chuyển sách. Học phần Kỹ thuật lập trình đóng vai trò nền tảng thiết yếu, giúp sinh viên làm quen và nắm vững các nguyên lý lập trình máy tính để giải quyết bài toán thực tế này.

- **Ngôn ngữ:** Chương trình được phát triển bằng ngôn ngữ Python.
- **Giao diện (Interface):** Thay vì sử dụng giao diện đồ họa (GUI), dự án này được thiết kế dưới dạng Giao diện dòng lệnh (Console/CLI). Lựa chọn này giúp hệ thống tối ưu hóa hoàn toàn tốc độ xử lý, loại bỏ chi phí tài nguyên đồ họa để tập trung trình diễn các cấu trúc dữ liệu chuyên sâu (Hash Map, Doubly Linked List, Queue) và các giải thuật phức tạp.
- **Lưu trữ dữ liệu:** Dữ liệu được lưu trữ bền vững dưới dạng các tệp tin CSV vật lý (`books.csv`, `users.csv`, `loans.csv`, `waiting_requests.csv`), cho phép dễ dàng kiểm soát, sao lưu và đọc/ghi liên tục theo thời gian thực.

2 Mô hình hóa bài toán

Yêu cầu bài toán: Xây dựng chương trình quản lý sách thư viện. Hệ thống sẽ tạo các menu chức năng để người dùng lựa chọn thao tác, nhập dữ liệu từ bàn phím và xuất kết quả trực quan ra màn hình. Bài toán được mô hình hóa thành 3 thực thể chính:

- **Sách (Book):** Quản lý định danh (Mã sách, Tên, Tác giả, Thể loại, Nhà xuất bản) và kiểm soát số lượng tồn kho theo thời gian thực.
- **Bạn đọc (Reader):** Lưu trữ thông tin cá nhân (Mã độc giả, Họ tên, Loại độc giả), phân quyền giới hạn số lượng và thời gian mượn.
- **Phiếu mượn (Loan):** Thực thể trung gian ràng buộc giữa Sách và Bạn đọc, ghi nhận chu kỳ: Ngày mượn, Hạn trả, Ngày trả thực tế, trạng thái và tự động tính toán phí phạt nếu trễ hạn.

Quy trình mượn/trả trong thực tế được mô phỏng khép kín: Người dùng đưa ra yêu cầu mượn → Hệ thống kiểm tra điều kiện nghiệp vụ (tồn kho, nợ xấu) → Cấp phát phiếu/Đưa vào hàng chờ → Nhận sách trả → Phạt trễ hạn → Cập nhật tồn kho và phục vụ hàng chờ.

3 Cấu trúc dự án

Dự án áp dụng mô hình Kiến trúc phân tầng (Layered Architecture) nhằm đảm bảo nguyên tắc Separation of Concerns (chia để trị), giúp mã nguồn độc lập, dễ gỡ lỗi và bảo trì:

- `objects/`: Lớp mô hình chứa các định nghĩa thực thể `Book`, `User`, `Loan`, `BorrowRequest`.
- `structure/`: Chứa các cấu trúc dữ liệu tự cài đặt từ con số 0 (Hash Map, Priority Queue, DLL, Waiting Queue).
- `storage/`: Tầng truy xuất, đảm nhiệm việc bóc tách dữ liệu từ RAM và ghi xuống tệp CSV.
- `logic/`: Tầng nghiệp vụ ("Bộ não"), xử lý các luồng: `loan_manager` (mượn/trả), `search` (tìm kiếm), `sort` (sắp xếp), `report` (thống kê).
- `interface/`: Tầng giao diện, chịu trách nhiệm in Menu ASCII phân trang, điều hướng luồng (`menu.py`) và kiểm duyệt dữ liệu thô (`validator.py`).
- `data/`: Thư mục chứa cơ sở dữ liệu vật lý (`.csv`).
- `main.py`: Điểm neo khởi chạy vòng lặp chương trình chính.

4 Mô tả sơ lược về chương trình

Để bảo mật dữ liệu, chương trình phân tách 2 phân hệ người dùng độc lập:

- **Quyền Quản trị viên (Admin):** Có toàn quyền kiểm soát. Admin thao tác thêm/sửa/xóa kho sách, quản lý thẻ độc giả, trực tiếp ghi nhận nghiệp vụ mượn/trả tại quầy và theo dõi báo cáo thống kê.
- **Quyền Bạn đọc (Reader):** Phân hệ cá nhân hóa (Chỉ đọc). Độc giả có thể tra cứu thư viện, xem danh sách sách đang mượn, kiểm tra phí phạt tạm tính và lịch sử cá nhân.

5 Mô tả chi tiết chương trình

Giao diện Console được tổ chức theo các Sub-menu. Mọi thao tác làm thay đổi dữ liệu đều kích hoạt ngầm hàm `_trigger_save()` để ghi file tức thời.

5.1 Quản lý Sách

Danh sách sách được hiển thị dưới dạng bảng ASCII phân trang (10 dòng/trang). Phía trên menu cung cấp các tác vụ:

- **Thao tác: Thêm sách mới**
  Hệ thống yêu cầu nhập từ bàn phím: Mã sách, Tên sách, Tác giả, Thể loại, NXB, Số lượng. Sau khi người dùng nhập, hệ thống sẽ chuyển qua module validator để kiểm tra:
  - Mã sách không được trùng lặp với mã đã có trong Hash Map.
  - Số lượng phải là số nguyên không âm.
  - Các trường bắt buộc không được bỏ trống.

  Trường hợp 1: Nếu thỏa mãn các điều kiện trên, sách mới được khởi tạo, đưa vào Bảng băm và thông báo thành công. Đồng thời ghi trực tiếp xuống `books.csv`.
  Trường hợp 2: Nếu có lỗi (trùng mã, sai định dạng), hệ thống lập tức báo lỗi đỏ `[!]`, chặn lưu dữ liệu và đưa người dùng trở lại sub-menu Quản lý Sách để thử lại.

- **Thao tác: Cập nhật thông tin**
  Nhập mã sách cần sửa. Hệ thống tra cứu mã trong Hash Map. Mã sách là khóa (key) nên bị khóa cứng, chỉ cho phép cập nhật Tên, Tác giả và Số lượng.
  Trường hợp 1: Nếu hợp lệ, hệ thống ghi đè thông tin. Đặc biệt, nếu số lượng mới lớn hơn số lượng cũ (tức là kho được bổ sung sách), hệ thống tự động gọi hàm kiểm tra Hàng chờ (`waiting_queue`) để cấp phát sách ngay cho những người đang đợi.

- **Thao tác: Xoá sách**
  Hệ thống yêu cầu mã sách. Trước khi xóa, module logic sẽ can thiệp kiểm tra điều kiện an toàn:
  - Quét toàn bộ Lịch sử giao dịch (DLL) xem có bất kỳ phiếu nào chứa mã sách này đang ở trạng thái "Đang mượn"hoặc "Quá hạn"hay không.

  Trường hợp 1: Nếu sách đang được mượn, hệ thống báo lỗi `[!]` Từ chối xóa và hủy thao tác.
  Trường hợp 2: Nếu an toàn, tiến hành xóa khỏi Hash Map và đồng bộ file CSV.

- **Thao tác: Tìm kiếm và Sắp xếp**
  Cho phép người dùng nhập từ khóa. Module tìm kiếm dùng thuật toán duyệt tuần tự chuỗi con (Substring matching) không phân biệt hoa thường. Danh sách kết quả được đưa qua thuật toán QuickSort để sắp xếp theo Tên sách (A-Z) trước khi đẩy lên bảng phân trang.

5.2 Quản lý Bạn đọc

- **Thao tác: Cấp thẻ bạn đọc mới**
  Hệ thống yêu cầu nhập Mã độc giả, Họ tên và Loại thẻ. Hệ thống kiểm duyệt sự tồn tại của mã.
  Trường hợp 1: Hợp lệ, hệ thống tạo đối tượng User mới, lưu vào mảng quản lý và ghi file `users.csv`.

- **Thao tác: Xem lịch sử giao dịch độc giả**
  Nhập mã độc giả. Hệ thống duyệt cấu trúc danh sách liên kết đôi (DLL) chứa toàn bộ giao dịch, lọc ra các phiếu khớp mã người dùng và in thành bảng thống kê cá nhân.

5.3 Mượn/Trả Sách

Đây là phân hệ phức tạp nhất, áp dụng luồng kiểm tra đa tầng.

- **Thao tác: Mượn sách**
  Nhập Mã độc giả và Mã sách. Hệ thống chạy luồng kiểm tra logic 7 bước nghiêm ngặt, theo đúng thứ tự, từ chối ngay khi gặp điều kiện không thỏa:

  1. Bạn đọc có tồn tại trong cơ sở dữ liệu không.
  2. Sách có tồn tại trong cơ sở dữ liệu không.
  3. Bạn đọc có đang nợ sách quá hạn không. Nếu có, từ chối phục vụ.
  4. Số lượng sách đang mượn có vượt giới hạn cho phép (`borrow_limit`) không.
  5. Bạn đọc có đang mượn chính cuốn sách này chưa (tránh mượn trùng).
  6. Sách có đang lưu hành không (`status == "active"`) - sách "inactive" bị từ chối.
  7. Tình trạng kho: Sách có `quantity > 0` không.

  Nghiệp vụ Hàng đợi (Waiting Queue): Nếu số lượng sách bằng 0, hệ thống đưa ra gợi ý: *Sách đã hết! Bạn có muốn vào hàng đợi không? (y/n).* Nếu chọn `y`, độc giả được đẩy vào cuối cấu trúc Queue chờ sách.
  Trường hợp Thành công: Nếu vượt qua toàn bộ 7 bước trên và kho có sẵn, hệ thống tạo Loan mới, tính `due_date` tự động, giảm tồn kho đi 1 và ghi lưu lịch sử.

- **Thao tác: Trả sách**
  Nhập Mã độc giả và Mã sách trả.

  1. Tra cứu Lịch sử (DLL) để tìm ra phiếu "Đang mượn"hoặc "Quá hạn"khớp dữ liệu.

2. Tính phí phạt: So sánh ngày trả với `due_date`. Nhân số ngày trễ với đơn giá phạt.
3. Đóng phiếu (cập nhật trạng thái thành "Đã trả") và tăng số lượng kho lên 1.

Cơ chế Ưu tiên Hàng chờ: Ngay khi sách được trả về kho, hệ thống tự động duyệt cấu trúc `WaitingQueue` theo đúng thứ tự đăng ký, tìm yêu cầu đầu tiên đang chờ đúng cuốn sách này (bỏ qua các yêu cầu của sách khác). Nếu người đó còn đủ điều kiện mượn (chưa nợ quá hạn, chưa vượt hạn mức), hệ thống rút yêu cầu khỏi hàng và tự động thiết lập một phiếu mượn mới cho họ; nếu không đủ điều kiện, người đó vẫn giữ chỗ trong hàng để được xét lại ở lượt sau, và hệ thống chuyển sang xét người kế tiếp khớp cùng cuốn sách. Sách chỉ "lọt"ra ngoài cho người tự do đến mượn khi không còn ai trong hàng đợi khớp đúng cuốn sách đó. Lập tức ghi dữ liệu xuống 4 file CSV.

II Cấu trúc dữ liệu tự cài đặt

Để đáp ứng yêu cầu học phần, nhóm không sử dụng trực tiếp các kiểu dữ liệu sẵn có của Python (`list`, `dict`, `set`, `queue.Queue`, hàm `hash()` built-in) trong tầng lưu trữ và xử lý nghiệp vụ. Toàn bộ dữ liệu của hệ thống — sách (`Books`), người dùng (`Users`), phiếu mượn/trả (`Loans`), hàng chờ mượn sách (`Queue`) — đều được tổ chức trên các cấu trúc dữ liệu do nhóm tự thiết kế và hiện thực tại thư mục `source_code/structure/`.

1 Mảng động tự cài đặt — CustomList

Vị trí: `source_code/structure/custom_list.py`

Đây là cấu trúc nền tảng của toàn bộ hệ thống, đóng vai trò thay thế cho `list` của Python. `CustomList` quản lý một mảng thô bên trong cùng hai biến trạng thái là dung lượng hiện có và số phần tử đang lưu. Khi số phần tử đạt đến dung lượng tối đa, cấu trúc tự động nhân đôi dung lượng và sao chép dữ liệu sang vùng nhớ mới — nhờ vậy phép `append()` đạt độ phức tạp $O(1)$ trung bình thay vì phải cấp phát lại sau mỗi lần thêm. Khi xóa một phần tử ở giữa mảng (`remove_at`), các phần tử phía sau được dồn lên một bậc để mảng luôn liền mạch.

Bên cạnh việc thêm/xóa, `CustomList` còn hỗ trợ tra cứu phần tử theo điều kiện tùy ý (`find`, `remove` nhận vào một hàm điều kiện — hàm này nhận một phần tử và trả về `True`/`False`). `CustomList` được tái sử dụng bên trong bốn cấu trúc khác: làm mảng bucket cho `BookHashMap`, làm vùng nhớ heap cho `PriorityQueue`, và làm kiểu dữ liệu trả về cho các phép truy vấn của `TransactionList`, `WaitingQueue`, `UserArray`.

Phép toán	**Độ phức tạp**
`append`	$O(1)$ trung bình
`get(index)`	$O(1)$
`remove_at` / `remove`(hàm điều kiện)	$O(n)$
`find`(hàm điều kiện)	$O(n)$

2 Danh sách liên kết kép — TransactionList

Vị trí: `source_code/structure/doubly_linked_list.py`

`TransactionList` lưu trữ toàn bộ lịch sử giao dịch mượn/trả sách dưới dạng danh sách liên kết kép, với mỗi nút mang theo dữ liệu giao dịch cùng hai con trỏ chỉ tới nút liền trước và liền sau. Cấu trúc duy trì đồng thời con trỏ đầu và cuối danh sách, cho phép thêm một

giao dịch mới (`add_transaction`) vào cuối với độ phức tạp $O(1)$ — phù hợp với đặc tính dữ liệu giao dịch luôn phát sinh và được ghi nhận theo trình tự thời gian.

Hai chức năng tra cứu quan trọng là `get_all_transactions()` — lấy toàn bộ lịch sử theo đúng thứ tự thời gian — và `get_transactions_by_user(user_id)` — lọc riêng giao dịch của một bạn đọc cụ thể. Cả hai đều phải duyệt tuần tự toàn bộ danh sách nên có độ phức tạp $O(n)$, một đánh đổi hợp lý vì danh sách liên kết không hỗ trợ truy cập ngẫu nhiên như mảng. Cấu trúc này được tầng lưu trữ (`storage/data_processor.py`) dùng để nạp/lưu lịch sử giao dịch, và được tầng nghiệp vụ (`logic/loan_manager.py`) dùng để ghi nhận mỗi phiếu mượn/trả mới.

Phép toán	**Độ phức tạp**
`add_transaction`	$O(1)$
`get_all_transactions` / `get_transactions_by_user`	$O(n)$

3 Bảng băm tự cài đặt — BookHashMap

Vị trí: `source_code/structure/hash_map.py`

Đây là cấu trúc dữ liệu phức tạp nhất trong hệ thống, được thiết kế theo kỹ thuật Separate Chaining (băm với danh sách liên kết tại mỗi ô) để lưu trữ và tra cứu sách theo mã sách với độ phức tạp trung bình $O(1)$. Hệ thống không dùng hàm `hash()` sẵn có của Python mà tự xây dựng một hàm băm đa thức, tính theo công thức `h = (h * 31 + ord(ch)) % capacity` lặp qua từng ký tự của mã sách. Mỗi bucket là một danh sách móc nối, cho phép nhiều khóa khác nhau cùng băm về một ô mà không làm mất dữ liệu; toàn bộ mảng bucket lại được lưu trên `CustomList` thay vì `list` Python, giữ tính nhất quán "tự cài đặt từ gốc" trong toàn hệ thống.

Ba phép toán `insert`, `search`, `delete` đều hoạt động theo cùng cơ chế: băm mã sách ra một chỉ số bucket, rồi duyệt chuỗi liên kết tại bucket đó để tìm đúng mục cần xử lý. Để giữ hiệu năng ổn định, cấu trúc theo dõi hệ số tải (số phần tử chia cho số bucket); khi hệ số này vượt 0.75, hàm `rehash` nội bộ tự động tăng gấp đôi số bucket và phân bổ lại toàn bộ phần tử — kỹ thuật chuẩn để tránh các chuỗi liên kết trở nên quá dài, làm suy biến độ phức tạp từ $O(1)$ về $O(n)$ trong trường hợp xấu nhất.

Phép toán	**Trung bình**	**Xấu nhất**
`insert` / `search` / `delete`	$O(1)$	$O(n)$
`Rehash` (khi vượt ngưỡng tải)	$O(n)$	$O(n)$

4 Hàng đợi có ưu tiên — PriorityQueue (Max-Heap nhị phân)

Vị trí: `source_code/structure/priority_queue.py`

Phục vụ riêng cho chức năng thống kê "Top sách được mượn nhiều nhất", nhóm hiện thực một heap nhị phân dạng max-heap, lưu trên `CustomList` theo cách biểu diễn cây nhị phân hoàn chỉnh bằng chỉ số mảng, không dùng con trỏ node. Tiêu chí so sánh giữa hai phần tử là số lượt mượn của sách — sách có lượt mượn cao nhất luôn nằm ở gốc heap. Khi thêm sách (`enqueue`), phần tử mới được đẩy lên đúng vị trí bằng cách so sánh liên tục với nút cha; khi lấy sách ra (`dequeue`), phần tử cuối heap được đưa lên gốc rồi hạ xuống đúng vị trí để khôi phục tính chất heap. Vì chiều cao của một heap nhị phân hoàn chỉnh luôn là $O(\log n)$, cả hai phép toán này đều đạt độ phức tạp $O(\log n)$. Riêng `peek_top_n(n)` — lấy ra n sách có lượt mượn cao nhất mà không phá hủy heap gốc — phải làm việc trên một bản sao của heap và lặp lại phép `dequeue` n lần, nên có độ phức tạp $O(n \log n)$.

Phép toán	**Độ phức tạp**
`enqueue` / `dequeue`	$O(\log n)$
`peek_top_n(n)`	$O(n \log n)$

5 Hàng đợi FIFO — WaitingQueue (Danh sách liên kết đơn)

Vị trí: `source_code/structure/waiting_queue.py`

Cấu trúc này giải quyết một bài toán nghiệp vụ cụ thể: khi một đầu sách hết bản sao khả dụng, bạn đọc được ghi nhận vào hàng chờ. Khi có sách được trả lại hoặc admin tăng số lượng tồn kho, nhóm sử dụng `remove_match` (xem bên dưới) để duyệt và rút đúng yêu cầu khớp `book_id` đang chờ, theo thứ tự đăng ký trước được xét trước (FIFO) trong số những người cùng chờ một cuốn sách - vì hàng đợi thực tế chứa lẫn yêu cầu của nhiều đầu sách khác nhau nên không thể dùng `dequeue` đơn giản (chỉ lấy đúng người đứng đầu toàn hàng, bất kể họ đang chờ sách nào).

Ngoài hai phép toán cốt lõi trên, `WaitingQueue` còn cho phép rút một yêu cầu cụ thể ra khỏi hàng giữa chừng thông qua `remove_match` mà vẫn giữ nguyên thứ tự các yêu cầu còn lại, với chi phí $O(n)$ do phải duyệt tìm.

Phép toán	**Độ phức tạp**
`enqueue` / `dequeue` / `peek`	$O(1)$
`remove_match`(hàm điều kiện)	$O(n)$

6 Mảng quản lý người dùng — UserArray

Vị trí: `source_code/structure/user_array.py`

`UserArray` là một lớp bọc quanh `CustomList`, được chuyên biệt hóa cho việc quản lý tài khoản bạn đọc. Thay vì để các tầng phía trên tự viết logic tra cứu, lớp này cung cấp sẵn `get_by_id` và `remove_by_id` — tìm và xóa người dùng theo đúng mã định danh nghiệp vụ — cả hai đều có độ phức tạp $O(n)$ vì phải duyệt tuyến tính để so khớp.

Phép toán	**Độ phức tạp**
`append`	$O(1)$ trung bình
`get_by_id` / `remove_by_id`	$O(n)$

7 Về các phương thức đặc biệt (Dunder methods)

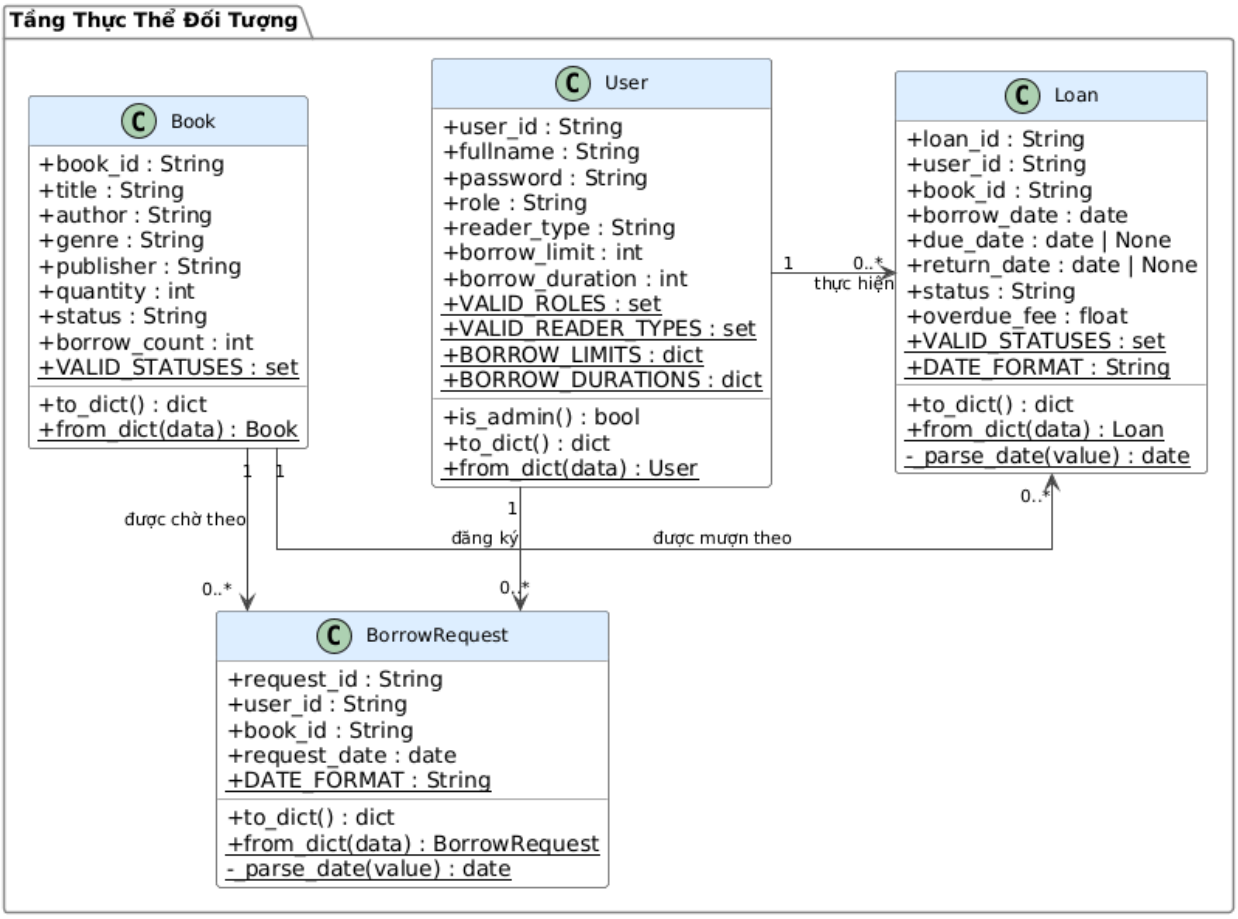
Một số cấu trúc còn cài đặt thêm các phương thức đặc biệt của Python — những phương thức có hai gạch dưới ở đầu và cuối tên, được Python tự gọi đến khi gặp cú pháp tương ứng:

Cú pháp viết	**Phương thức được gọi**
`len(obj)`	`__len__`
`for x in obj:`	`__iter__`
`obj[i]`	`__getitem__`
`a == b`	`__eq__`
`print(obj)`	`__repr__`

Nhờ vậy, các cấu trúc tự viết dùng được trực tiếp với cú pháp Python gốc (`for book in book_list`, `len(transactions)`...) giống như một `list` built-in. Cách đặt tên hai gạch dưới là quy ước riêng Python dành cho nhóm phương thức này, nên các phương thức do nhóm tự viết như `append`, `enqueue`, `search` đều giữ tên thường, tránh trùng với quy ước đó.

III Cấu trúc đối tượng

Trước khi đi vào trình bày chi tiết từng cấu trúc đối tượng, nhóm xin giới thiệu sơ đồ lớp UML (Class Diagram) thể hiện tổng quan các thuộc tính, phương thức và mối quan hệ giữa bốn đối tượng cốt lõi của hệ thống: `User`, `Loan`, `BorrowRequest` và `Book`. Sơ đồ này giúp hình dung một cách trực quan cách các đối tượng liên kết với nhau thông qua các trường khóa ngoại (`user_id`, `book_id`), trước khi đi sâu vào cơ chế hoạt động của từng phương thức trong các mục tiếp theo.



1 Cấu trúc "Book"

Đại diện cho cấu trúc thông tin của một đầu sách trong thư viện.

Các thuộc tính của đối tượng `Book` bao gồm:

Thuộc tính	Mô tả	Kiểu dữ liệu
book_id	Mã định danh duy nhất của sách	str
title	Tên sách	str
author	Tên tác giả	str
genre	Thể loại sách	str
publisher	Nhà xuất bản	str
quantity	Số lượng sách hiện có trong thư viện	int
status	Tình trạng vật lý của sách (active/inactive)	str
borrow_count	Tổng số lượt sách đã được mượn	int

1.1 Khởi tạo thông tin sách

Hàm khởi tạo (constructor), được tự động gọi khi một đối tượng `Book` mới được tạo.

- **Input:** Tham số đầu vào `self` cùng các thông tin về sách: mã sách, tên sách, tác giả, thể loại, nhà xuất bản, số lượng, tình trạng và số lượt mượn.
- **Output:** Không có kết quả đầu ra.
- **Code sử dụng:**

```
def __init__(
  self,
  book_id: str,
  title: str,
  author: str,
  genre: str,
  publisher: str,
  quantity: int = 1,
  status: str = "active",
  borrow_count: int = 0,
):
```

- **Cơ chế hoạt động:** Gán trực tiếp các giá trị từ input vào các thuộc tính tương ứng của `self`. Năm thuộc tính đầu (`book_id`, `title`, `author`, `genre`, `publisher`) bắt buộc phải nhập vì là thông tin cốt lõi để nhận diện một đầu sách. Ba thuộc tính còn lại (`quantity`, `status`, `borrow_count`) có giá trị mặc định, cho phép khởi tạo nhanh một sách mới nhập về mà chưa cần điền đầy đủ.

1.2 Chuyển dạng đối tượng

Định nghĩa cách đối tượng `Book` được hiển thị dưới dạng một chuỗi ký tự, hữu ích cho việc in thông tin ra để gỡ lỗi (debug).

- **Input:** Tham số đầu vào `self`.
- **Output:** Chuỗi dễ đọc gom các thuộc tính của sách.
- **Code sử dụng:**

```
def __repr__(self) -> str:
```

- **Cơ chế hoạt động:** Chuyển đổi thông tin mã sách, tên sách, tác giả, thể loại, nhà xuất bản, số lượng, tình trạng và số lượt mượn thành một chuỗi duy nhất theo định dạng `Book(book_id=..., title=..., ...)`, giúp người phát triển kiểm tra nhanh giá trị một đối tượng `Book` ngay trên màn hình console.

1.3 Sao chép dữ liệu

Chuyển đổi một đối tượng `Book` thành một cấu trúc dữ liệu cơ bản hơn (dictionary) để dễ dàng lưu trữ, cụ thể là ghi xuống file `books.csv`.

- **Input:** Tham số đầu vào `self`.
- **Output:** Một dictionary (`dict`) chứa toàn bộ 8 cặp khóa-giá trị tương ứng với thuộc tính của đối tượng.
- **Code sử dụng:**

```
def to_dict(self) -> dict:
```

- **Cơ chế hoạt động:** Đóng gói toàn bộ thuộc tính của `self` thành một `dict` với khóa là tên thuộc tính. Dict trả về được `storage.data_processor` sử dụng trực tiếp để ghi từng dòng dữ liệu xuống file `books.csv`.

1.4 Giải nén dữ liệu

Phương thức của lớp (`@classmethod`), có chức năng ngược lại với `to_dict`. Tạo ra một đối tượng `Book` mới từ một dictionary đọc được từ file CSV.

- **Input:** Lớp `Book`, một `data` tương ứng với một dòng dữ liệu đọc từ file `books.csv`.
- **Output:** Trả về một đối tượng `Book` mới, khởi tạo với dữ liệu từ `data`.

- **Code sử dụng:**

```
@classmethod
def from_dict(cls, data: dict) -> "Book":
```

- **Cơ chế hoạt động:** Vì dữ liệu đọc từ CSV luôn ở dạng chuỗi, hàm thực hiện ép kiểu an toàn: `book_id`, `title`, `author`, `genre`, `publisher`, `status` ép về `str`; `quantity`, `borrow_count` ép về `int`. Dùng `data.get(key, default)` để lấy giá trị mặc định nếu dòng dữ liệu thiếu trường (`quantity`, `status`, `borrow_count`), tránh chương trình lỗi `KeyError` khi đọc file CSV không đầy đủ. Trả về đối tượng `Book` mới với đầy đủ thông tin vừa giải nén.

1.5 So sánh đối tượng

Định nghĩa cách hai đối tượng `Book` được so sánh bằng nhau với nhau thông qua phép toán ==.

- **Input:** Tham số đầu vào `self` và `other` — đối tượng cần so sánh.

- **Output:** Trả về `True` nếu hai sách có cùng `book_id`, `False` nếu khác, hoặc `NotImplemented` nếu `other` không phải là một đối tượng `Book`.

- **Code sử dụng:**

```
def __eq__(self, other: object) -> bool:
```

- **Cơ chế hoạt động:** Kiểm tra `other` có phải là instance của `Book` không. Nếu không, trả về `NotImplemented` - đây là quy ước chuẩn của Python, cho phép Python tự thử so sánh theo chiều ngược lại trước khi báo lỗi. Nếu `other` là một `Book`, so sánh duy nhất qua `book_id` - vì đây là khóa định danh duy nhất của một đầu sách trong toàn hệ thống, hai sách có cùng `book_id` đương nhiên là cùng một đầu sách, bất kể các thuộc tính khác (số lượng, tình trạng...) có khác nhau hay không tại từng thời điểm. Phương thức này được `logic.search` sử dụng để xác định một sách có khớp với kết quả tìm kiếm hay không.

2 Cấu trúc "User"

Đại diện cho cấu trúc thông tin của một tài khoản người dùng trong hệ thống thư viện, bao gồm cả thủ thư (Admin) và bạn đọc (sinh viên/giảng viên).

Các thuộc tính của đối tượng `User` được trình bày chi tiết trong bảng sau:

Thuộc tính	Mô tả	Kiểu dữ liệu
`user_id`	Mã định danh duy nhất (MSSV, mã giảng viên, hoặc "admin")	`str`
`fullname`	Họ và tên đầy đủ	`str`
`password`	Mật khẩu tài khoản	`str`
`role`	Vai trò hệ thống (admin/user)	`str`
`reader_type`	Loại bạn đọc ("student", "lecturer", hoặc rỗng nếu là admin)	`str`
`borrow_limit`	Hạn mức mượn tối đa	`int`
`borrow_duration`	Số ngày được mượn	`int`

2.1 Khởi tạo thông tin người đọc

Được gọi tự động khi một đối tượng `User` mới được tạo.

- **Input:** Tham số đầu vào `self` cùng các thông tin: mã người dùng, họ tên, mật khẩu, vai trò và loại bạn đọc.
- **Output:** Không có kết quả đầu ra.
- **Code sử dụng:**

```
def __init__(
  self,
  user_id: str,
  fullname: str,
  password: str,
  role: str = "user",
  reader_type: str = "student",
):
```

- **Cơ chế hoạt động:** Gán trực tiếp năm tham số đầu vào vào các thuộc tính tương ứng của `self`. Tự động tính `borrow_limit` bằng cách tra cứu giá trị `reader_type` vừa nhận trong hằng số `BORROW_LIMITS = {"student": 3, "lecturer": 5, "": 0}`. Tự động tính `borrow_duration` tương tự, tra cứu trong `BORROW_DURATIONS = {"student": 14, "lecturer": 21, "": 0}`. Cách thiết kế này đóng gói trực tiếp hai luật nghiệp vụ "sinh viên được mượn tối đa 3 cuốn, mỗi cuốn được mượn trong 14 ngày, giảng viên tối

đa 5 cuốn, mỗi cuốn được mượn trong 21 ngày" ngay tại tầng dữ liệu. Nhờ vậy, mọi nơi sử dụng đối tượng `User` (ví dụ `logic.loan_manager`) chỉ cần đọc `user.borrow_limit` mà không cần tính toán lại, tránh lặp code và đảm bảo tính nhất quán trên toàn hệ thống.

2.2 Chuyển dạng đối tượng

Định nghĩa cách đối tượng `User` được hiển thị dưới dạng một chuỗi ký tự, hữu ích cho việc gỡ lỗi (debug).

- **Input:** Tham số đầu vào `self`.
- **Output:** Chuỗi dễ đọc gom các thuộc tính của tài khoản.
- **Code sử dụng:**

```
def __repr__(self) -> str:
```

- **Cơ chế hoạt động:** Chuyển đổi thông tin mã người dùng, họ tên, vai trò, loại bạn đọc, hạn mức và số ngày mượn thành một chuỗi duy nhất theo định dạng `User(user_id=..., fullname=..., ...)`. Lưu ý hàm không hiển thị `password` trong chuỗi trả về, hạn chế rủi ro lộ thông tin nhạy cảm khi in đối tượng ra console phục vụ gỡ lỗi.

2.3 Lấy dict (khóa, giá trị) nội bộ của đối tượng

Chuyển đổi một đối tượng `User` thành một dictionary để dễ dàng lưu trữ, cụ thể là ghi xuống file `users.csv`.

- **Input:** Tham số đầu vào `self`.
- **Output:** Một dictionary (`dict`) chứa 5 cặp khóa-giá trị tương ứng với thuộc tính cốt lõi của đối tượng.
- **Code sử dụng:**

```
def to_dict(self) -> dict:
```

- **Cơ chế hoạt động:** Đóng gói 5 thuộc tính `user_id`, `fullname`, `password`, `role`, `reader_type` thành một `dict`. Đáng chú ý, `borrow_limit` và `borrow_duration` không được đưa vào dict này — vì hai giá trị này không phải dữ liệu gốc mà chỉ là kết quả tính toán tự động từ `reader_type` ngay trong `__init__`. Lưu thêm hai giá trị này xuống CSV là dư thừa, và nếu sau này nghiệp vụ thay đổi hạn mức thì dữ liệu cũ trong CSV sẽ không tự cập nhật theo, gây sai lệch.

2.4 Giải nén dữ liệu

Phương thức của lớp (`@classmethod`), có chức năng ngược lại với `to_dict`. Tạo ra một đối tượng `User` mới từ một dictionary đọc được từ file CSV.

- **Input:** Lớp `User`, một `data` tương ứng với một dòng dữ liệu đọc từ file `users.csv`.
- **Output:** Trả về một đối tượng `User` mới, khởi tạo với dữ liệu từ `data`.
- **Code sử dụng:**

```
@classmethod
def from_dict(cls, data: dict) -> "User":
```

- **Cơ chế hoạt động:** Ép kiểu an toàn cả 5 trường về `str` trước khi truyền vào constructor. Dùng `data.get("role", "user")` và `data.get("reader_type", "student")` để tự điền giá trị mặc định nếu dòng dữ liệu thiếu trường, tránh lỗi `KeyError` khi file CSV không đầy đủ. Vì constructor tự tính lại `borrow_limit` và `borrow_duration` từ `reader_type`, đối tượng `User` được tạo ra luôn có hạn mức đúng theo luật nghiệp vụ hiện tại — kể cả khi dữ liệu được nạp từ một file CSV cũ.

2.5 Kiểm tra vai trò

Hàm hỗ trợ kiểm tra nhanh một tài khoản có quyền quản trị hay không.

- **Input:** Tham số đầu vào `self`.
- **Output:** Trả về `True` nếu tài khoản có `role` là "admin", `False` nếu ngược lại.
- **Code sử dụng:**

```
def is_admin(self) -> bool:
```

- **Cơ chế hoạt động:** So sánh trực tiếp `self.role` với chuỗi "admin". Hàm này được `main.py` gọi ngay sau khi đăng nhập thành công để quyết định phân luồng vào `menu.run_admin_menu()` hay `menu.run_user_menu()`.

2.6 So sánh đối tượng

Định nghĩa cách hai đối tượng `User` được so sánh bằng nhau với nhau thông qua phép toán ==.

- **Input:** Tham số đầu vào `self` và `other`.

- **Output:** Trả về `True` nếu hai tài khoản có cùng `user_id`, `False` nếu khác, hoặc `NotImplemented` nếu `other` không phải `User`.

- **Code sử dụng:**

```
def __eq__(self, other: object) -> bool:
```

- **Cơ chế hoạt động:** Kiểm tra `other` có phải instance của `User` không, nếu không trả về `NotImplemented` theo quy ước chuẩn của Python. So sánh duy nhất qua `user_id` — vì đây là khóa định danh duy nhất của một tài khoản, không phụ thuộc vào các thông tin khác như họ tên hay mật khẩu có trùng nhau hay không.

3 Cấu trúc "Loan"

Đại diện cho cấu trúc thông tin của một phiếu giao dịch mượn/trả sách trong thư viện. Các thuộc tính của đối tượng `Loan` được trình bày chi tiết trong bảng sau:

Thuộc tính	**Mô tả**	**Kiểu dữ liệu**
`loan_id`	Mã phiếu mượn duy nhất	`str`
`user_id`	Mã bạn đọc thực hiện mượn	`str`
`book_id`	Mã sách được mượn	`str`
`borrow_date`	Ngày mượn sách	`date`
`due_date`	Ngày hết hạn trả	`date` hoặc `None`
`return_date`	Ngày trả thực tế (nếu chưa trả sẽ là None)	`date` hoặc `None`
`status`	Trạng thái phiếu ("borrowing", "returned" hoặc "overdue")	`str`
`overdue_fee`	Tiền phạt quá hạn (VNĐ)	`float`

3.1 Khởi tạo thông tin mượn/trả sách

Được gọi tự động khi một đối tượng `Loan` mới được tạo.

- **Input:** Tham số đầu vào `self` cùng các thông tin: mã phiếu, mã bạn đọc, mã sách, ngày mượn, ngày hết hạn, ngày trả, trạng thái và tiền phạt.

- **Output:** Không có kết quả đầu ra.

- **Code sử dụng:**

```
def __init__(
  self,
  loan_id: str,
  user_id: str,
  book_id: str,
  borrow_date: date,
  due_date: Optional[date] = None,
  return_date: Optional[date] = None,
  status: str = "borrowing",
  overdue_fee: float = 0.0,
):
```

- **Cơ chế hoạt động:** Gán trực tiếp tám tham số đầu vào vào các thuộc tính tương ứng của `self`. Khác với `User` (tự tính `borrow_limit` ngay trong constructor), `Loan` không tự tính `due_date` — tham số này mặc định là `None` và được để ngỏ cho `logic.loan_manager` tính toán (`due_date = borrow_date + user.borrow_duration`) tại thời điểm xử lý nghiệp vụ mượn sách. Cách thiết kế này tách biệt rõ tầng dữ liệu (Object Layer) khỏi tầng nghiệp vụ (Logic Layer): `Loan` chỉ có nhiệm vụ lưu trữ thông tin, còn quy tắc "hạn trả được tính như thế nào"hoàn toàn thuộc về `loan_manager`. Nếu nghiệp vụ tính hạn trả thay đổi trong tương lai, chỉ cần sửa ở `loan_manager` mà không cần đụng đến `Loan`.

3.2 Chuyển dạng đối tượng

Định nghĩa cách đối tượng `Loan` được hiển thị dưới dạng một chuỗi ký tự, hữu ích cho việc gỡ lỗi (debug).

- **Input:** Tham số đầu vào `self`.

- **Output:** Chuỗi dễ đọc gom các thuộc tính của phiếu mượn.

- **Code sử dụng:**

```
def __repr__(self) -> str:
```

- **Cơ chế hoạt động:** Chuyển đổi thông tin mã phiếu, mã bạn đọc, mã sách, ngày mượn, ngày hết hạn, ngày trả, trạng thái và tiền phạt thành một chuỗi duy nhất theo định dạng `Loan(loan_id=..., user_id=..., ...)`.

3.3 Lấy dict (khóa, giá trị) nội bộ của đối tượng

Chuyển đổi một đối tượng `Loan` thành một dictionary để dễ dàng lưu trữ, cụ thể là ghi xuống file `loans.csv`.

- **Input:** Tham số đầu vào `self`.
- **Output:** Một dictionary (`dict`) chứa toàn bộ 8 cặp khóa-giá trị tương ứng với thuộc tính của đối tượng.
- **Code sử dụng:**

```
def to_dict(self) -> dict:
```

- **Cơ chế hoạt động:** Định nghĩa một hàm nội bộ `format_date` để xử lý ba trường kiểu ngày (`borrow_date`, `due_date`, `return_date`): nếu giá trị là `None` hoặc rỗng thì trả về chuỗi rỗng `""`, nếu đã là chuỗi thì giữ nguyên, còn lại thì gọi `strftime` để chuyển đối tượng `date` thành chuỗi theo định dạng chuẩn `"%Y-%m-%d"`. Áp dụng `format_date` cho cả ba trường ngày trước khi đóng gói toàn bộ tám thuộc tính vào dict trả về — đảm bảo dữ liệu ghi xuống CSV luôn là chuỗi văn bản nhất quán, không bị lỗi khi `due_date` hoặc `return_date` đang là `None` (trường hợp phiếu chưa trả).

3.4 Giải nén dữ liệu

Phương thức của lớp (`@classmethod`), có chức năng ngược lại với `to_dict`. Tạo ra một đối tượng `Loan` mới từ một dictionary đọc được từ file CSV.

- **Input:** Lớp `Loan`, một `data` tương ứng với một dòng dữ liệu đọc từ file `loans.csv`.
- **Output:** Trả về một đối tượng `Loan` mới, khởi tạo với dữ liệu từ `data`.
- **Code sử dụng:**

```
@classmethod
def from_dict(cls, data: dict) -> "Loan":
```

- **Cơ chế hoạt động:** Ép kiểu an toàn `loan_id`, `user_id`, `book_id` về `str`; `overdue_fee`, `status` về `float`. Ba trường ngày (`borrow_date`, `due_date`, `return_date`) được chuyển đổi thông qua một hàm phụ trợ riêng `_parse_date`, không ép kiểu trực tiếp như các trường khác. Dùng `data.get("status", "borrowing")` và `data.get("overdue_fee", 0.0)` để tự điền giá trị mặc định nếu dòng dữ liệu thiếu trường.

3.5 Chuyển đổi định dạng ngày

Phương thức phụ trợ nội bộ (`@classmethod`), chuyên trách việc chuyển đổi giá trị ngày từ nhiều định dạng đầu vào khác nhau về thống nhất kiểu `date` của Python.

- **Input:** Lớp `Loan`, một giá trị `value` có thể là chuỗi ISO, đối tượng `date`, đối tượng `datetime`, hoặc rỗng.
- **Output:** Trả về đối tượng `date`, hoặc `None` nếu đầu vào rỗng.
- **Code sử dụng:**

```
@classmethod
def _parse_date(cls, value):
```

- **Cơ chế hoạt động:** Nếu `value` là `None` hoặc chuỗi rỗng, trả về `None` ngay (trường hợp phiếu chưa có `due_date` hoặc chưa trả). Nếu `value` đã là đối tượng `date`, trả về nguyên giá trị đó. Nếu `value` là đối tượng `datetime`, gọi `.date()` để lấy phần ngày, bỏ phần giờ. Nếu `value` là chuỗi (trường hợp phổ biến nhất khi đọc từ CSV), dùng `datetime.strptime(value, cls.DATE_FORMAT).date()` để phân tích chuỗi theo định dạng "%Y-%m-%d"rồi chuyển về `date`. Việc tách `_parse_date` thành một hàm riêng (bắt đầu bằng dấu gạch dưới, quy ước của Python cho hàm nội bộ) cho phép `from_dict` gọi lại ba lần cho ba trường ngày khác nhau mà không lặp code.

3.6 So sánh đối tượng

Định nghĩa cách hai đối tượng `Loan` được so sánh bằng nhau với nhau thông qua phép toán ==.

- **Input:** Tham số đầu vào `self` và `other`.
- **Output:** Trả về `True` nếu hai phiếu mượn có cùng `loan_id`, `False` nếu khác, hoặc `NotImplemented` nếu `other` không phải `Loan`.
- **Code sử dụng:**

```
def __eq__(self, other: object) -> bool:
```

- **Cơ chế hoạt động:** Kiểm tra `other` có phải instance của `Loan` không, nếu không trả về `NotImplemented`. So sánh duy nhất qua `loan_id` — khóa định danh duy nhất của một phiếu mượn trong toàn hệ thống.

4 Cấu trúc "BorrowRequest"

Đại diện cho cấu trúc thông tin của một yêu cầu chờ mượn sách, được tạo ra khi một bạn đọc muốn mượn một cuốn sách đã hết trong kho.

Các thuộc tính của đối tượng `BorrowRequest` được trình bày chi tiết trong bảng sau:

Thuộc tính	**Mô tả**	**Kiểu dữ liệu**
`request_id`	Mã yêu cầu chờ duy nhất	`str`
`user_id`	Mã bạn đọc đang chờ	`str`
`book_id`	Mã sách đang chờ	`str`
`request_date`	Ngày đăng ký chờ	`date`

4.1 Khởi tạo thông tin yêu cầu chờ

Được gọi tự động khi một đối tượng `BorrowRequest` mới được tạo.

- **Input:** Tham số đầu vào `self` cùng các thông tin: mã yêu cầu, mã bạn đọc, mã sách và ngày đăng ký chờ.
- **Output:** Không có kết quả đầu ra.
- **Code sử dụng:**

```
def __init__(
  self,
  request_id: str,
  user_id: str,
  book_id: str,
  request_date: date,
):
```

- **Cơ chế hoạt động:** Gán trực tiếp bốn tham số đầu vào vào các thuộc tính tương ứng của `self`. Đây là object có cấu trúc đơn giản nhất trong bốn object của hệ thống, vì nhiệm vụ duy nhất là ghi lại "ai đang chờ mượn sách gì, từ ngày nào"— không cần lưu trạng thái hay tính toán gì thêm. Một yêu cầu chờ chỉ tồn tại trong `WaitingQueue` cho đến khi được `logic.loan_manager` xử lý (rút khỏi hàng đợi khi mượn được sách).

4.2 Chuyển dạng đối tượng

Định nghĩa cách đối tượng `BorrowRequest` được hiển thị dưới dạng một chuỗi ký tự, hữu ích cho việc gỡ lỗi (debug).

- **Input:** Tham số đầu vào `self`.

- **Output:** Chuỗi dễ đọc gom các thuộc tính của yêu cầu chờ.

- **Code sử dụng:**

```
def __repr__(self) -> str:
```

- **Cơ chế hoạt động:** Chuyển đổi thông tin mã yêu cầu, mã bạn đọc, mã sách và ngày đăng ký chờ thành một chuỗi duy nhất theo định dạng `BorrowRequest(request_id=..., user_id=..., ...)`.

4.3 Lấy dict (khóa, giá trị) nội bộ của đối tượng

Chuyển đổi một đối tượng `BorrowRequest` thành một dictionary để dễ dàng lưu trữ, cụ thể là ghi xuống file `waiting_requests.csv`.

- **Input:** Tham số đầu vào `self`.

- **Output:** Một dictionary (`dict`) chứa 4 cặp khóa-giá trị tương ứng với thuộc tính của đối tượng.

- **Code sử dụng:**

```
def to_dict(self) -> dict:
```

- **Cơ chế hoạt động:** Kiểm tra `request_date` có phải instance của `date` không; nếu đúng thì gọi `strftime(self.DATE_FORMAT)` để chuyển thành chuỗi, nếu không (trường hợp hiếm, dữ liệu lỗi) thì trả về chuỗi rỗng `""`để tránh chương trình bị crash. Đóng gói cả bốn thuộc tính thành dict trả về.

4.4 Giải nén dữ liệu

Phương thức của lớp (`@classmethod`), có chức năng ngược lại với `to_dict`. Tạo ra một đối tượng `BorrowRequest` mới từ một dictionary đọc được từ file CSV.

- **Input:** Lớp `BorrowRequest`, một `data` tương ứng với một dòng dữ liệu đọc từ file `waiting_requests.csv`.

- **Output:** Trả về một đối tượng `BorrowRequest` mới, khởi tạo với dữ liệu từ `data`.

- **Code sử dụng:**

```
@classmethod
def from_dict(cls, data: dict) -> "BorrowRequest":
```

- **Cơ chế hoạt động:** Ép kiểu an toàn `request_id`, `user_id`, `book_id`, về `str`. Trường `request_date` được chuyển đổi thông qua hàm phụ trợ `_parse_date`, cùng cơ chế và cùng định dạng "%Y-%m-%d"như object `Loan` đã sử dụng — thể hiện sự nhất quán trong cách xử lý ngày tháng giữa các object của hệ thống.

4.5 So sánh đối tượng

Định nghĩa cách hai đối tượng `BorrowRequest` được so sánh bằng nhau với nhau thông qua phép toán ==.

- **Input:** Tham số đầu vào `self` và `other`.
- **Output:** Trả về `True` nếu hai yêu cầu chờ có cùng `request_id`, `False` nếu khác, hoặc `NotImplemented` nếu `other` không phải `BorrowRequest`.
- **Code sử dụng:**

```
def __eq__(self, other: object) -> bool:
```

- **Cơ chế hoạt động:** Kiểm tra `other` có phải instance của `BorrowRequest` không, nếu không trả về `NotImplemented`. So sánh duy nhất qua `request_id` - khóa định danh duy nhất của một yêu cầu chờ.

IV Logic nghiệp vụ (Thuật toán xử lý)

Nếu tầng structure ở trên là nơi lưu trữ dữ liệu, thì tầng logic (`source_code/logic/`) là nơi xử lý dữ liệu đó để tạo ra kết quả nghiệp vụ — tìm sách, sắp xếp danh mục, xét mượn/trả, tính tiền phạt, gom báo cáo thống kê. Các hàm trong tầng này được quy định không chứa lệnh `input()`/`print()`, chỉ nhận dữ liệu vào và trả kết quả ra qua `return`, nhờ vậy có thể tái sử dụng và kiểm thử (viết test) độc lập với màn hình giao diện.

1 Tìm kiếm sách — `search.py`

Vị trí: `source_code/logic/search.py`

Việc tìm sách được chia thành hai kiểu. Tìm chính xác theo mã sách (`search_by_id`) chỉ đơn giản là gọi lại phép search của `BookHashMap`, nên đạt độ phức tạp $O(1)$. Ngược lại, tìm theo tên sách, tác giả, thể loại hoặc nhà xuất bản (`search_by_title`, `search_by_author`, `search_by_genre`, `search_by_publisher`) là tìm kiếm tương đối — chấp nhận sách khớp một phần với từ khóa, không phân biệt hoa/thường — nên cả bốn hàm đều phải duyệt qua toàn bộ sách trong `BookHashMap` và so khớp từng cuốn.

Hàm	**Độ phức tạp**
`search_by_id`	$O(1)$

`search_by_title`
`search_by_author`
`search_by_genre`
`search_by_publisher`
	$O(n)$

2 Sắp xếp danh mục — Quick Sort tự cài (`sort.py`)

Vị trí: `source_code/logic/sort.py`

Để hiển thị danh mục sách theo thứ tự A-Z, nhóm tự cài giải thuật Quick Sort thay vì dùng `sorted()`/`list.sort()` có sẵn của Python. Giải thuật chọn phần tử cuối đoạn đang xét làm chốt (pivot), phân hoạch (`_partition`) bằng cách dồn các phần tử có khóa nhỏ hơn hoặc bằng chốt về bên trái, các phần tử lớn hơn về bên phải, rồi đặt chốt vào đúng vị trí giữa hai bên. Sau đó, hàm tiếp tục gọi lại chính nó cho từng nửa đoạn còn lại, đến khi mỗi đoạn chỉ còn một phần tử.

Điểm đáng nói là phần lõi chia-để-trị (`_quick_sort_recursive`) không tự biết đang so sánh theo tiêu chí nào — nó chỉ gọi một hàm lấy khóa so sánh, gọi là `key_fn`, được truyền vào từ bên ngoài để lấy ra giá trị cần so sánh từ một cuốn sách. Hàm `quicksort(book_list, key="title")` nhận vào một tham số `key` để chọn tiêu chí sắp xếp, mặc định là "title" (tên

sách); ngoài ra còn hỗ trợ "author" (tác giả) và "publisher" (nhà xuất bản) — mỗi tiêu chí được ánh xạ sang một `key_fn` riêng tương ứng.

Với tiêu chí "title", `key_fn(book)` chỉ trả về tên sách (viết thường) để so sánh trực tiếp. Với "author" hoặc "publisher", `key_fn(book)` trả về một tuple gồm hai giá trị — ví dụ (tác giả, tên sách) — để khi hai cuốn sách có cùng tác giả, Python sẽ tự so tiếp phần tử thứ hai của tuple (tên sách) nhằm sắp xếp phụ theo alphabet, tránh thứ tự bị xáo trộn ngẫu nhiên giữa các sách cùng tác giả. Nhờ thiết kế này, đúng một đoạn code chia-để-trị có thể dùng lại cho cả ba tiêu chí, chỉ cần đổi `key_fn` truyền vào.

Trường hợp	**Độ phức tạp**
Trung bình	$O(n \log n)$
Xấu nhất (mảng đã gần có thứ tự)	$O(n^2)$

3 Điều phối mượn/trả sách — `loan_manager.py`

Vị trí: `source_code/logic/loan_manager.py`

Đây là mô-đun phức tạp nhất về nghiệp vụ, gồm ba luồng chính.

a) Xử lý mượn sách — `process_borrow`: Kiểm tra tuần tự theo đúng thứ tự sau, từ chối ngay khi gặp điều kiện không thỏa:

1. Bạn đọc (`user_id`) có tồn tại không.
2. Sách (`book_id`) có tồn tại không.
3. Bạn đọc có phiếu nào đang ở trạng thái "overdue" không.
4. Số phiếu đang ở trạng thái "borrowing"/"overdue" của bạn đọc đã đạt `borrow_limit` chưa.
5. Bạn đọc có đang mượn chính cuốn sách này (`status` "borrowing"/"overdue") không.
6. Sách có `status == "active"` không (sách "inactive" thì từ chối).
7. Sách còn `quantity > 0` trong kho không.

Nếu qua hết bảy bước trên: tạo một phiếu mượn mới (`Loan`) với `due_date` tính từ `borrow_date + borrow_duration` của bạn đọc, thêm phiếu vào `TransactionList`, giảm `quantity` đi 1 và tăng `borrow_count` lên 1.

b) Xử lý trả sách — `process_return`:

1. Kiểm tra bạn đọc và sách có tồn tại không.

2. Tìm phiếu mượn gần nhất của đúng bạn đọc và sách, đang ở trạng thái "borrowing" hoặc "overdue".
3. Nếu `return_date` sau `due_date`: tính tiền phạt = số ngày trễ × đơn giá phạt theo `reader_type`.
4. Cập nhật phiếu: `return_date`, `status = "returned"`, `overdue_fee`.
5. Tăng `quantity` của sách lên 1.
6. Gọi `serve_waiting_queue` để xét ngay cho người trong hàng chờ.

c) Ưu tiên hàng chờ — `serve_waiting_queue`: Mỗi khi `quantity` của một sách tăng lên (do bước 5 ở trên, hoặc do admin sửa thông tin sách để tăng kho), hàm này lặp lại như sau cho đến khi hết hàng hoặc hết người chờ khớp:

1. Duyệt hàng chờ theo đúng thứ tự đăng ký (FIFO), bỏ qua các yêu cầu của sách khác.
2. Gặp yêu cầu đầu tiên đúng `book_id`: gọi thử `process_borrow` cho người đó.
3. Nếu mượn được: rút yêu cầu khỏi hàng chờ, dừng lượt xét hiện tại (vòng lặp ngoài sẽ kiểm tra lại `quantity`, lặp tiếp nếu còn sách).
4. Nếu không đủ điều kiện (ví dụ đang nợ "overdue"): bỏ qua, người đó vẫn giữ chỗ trong hàng chờ, xét sang người kế tiếp.
5. Nếu không ai trong hàng đợi khớp `book_id`: dừng, sách ở lại kho cho ai đến mượn tự do.

Ngoài ba luồng trên, mô-đun còn có `calculate_overdue_fee` (tính phạt tạm tính cho phiếu chưa trả, phục vụ hiển thị), `is_book_on_loan` (kiểm tra sách có đang được mượn trước khi cho admin xóa), và `add_to_waiting_queue` (ghi nhận yêu cầu chờ khi hết kho).

Hàm	**Độ phức tạp**
`process_borrow`	$O(n)$
`process_return`	$O(n)$
`serve_waiting_queue`	$O(k \times n)$ — với k là số người trong hàng chờ được xét
`calculate_overdue_fee` / `is_book_on_loan`	$O(n)$

Lưu ý (Sửa lỗi logic): Trong quá trình phát triển, nhóm phát hiện ra rằng ban đầu hàng chờ chỉ được xét lại khi có người trả sách. Điều này dẫn đến lỗi: nếu admin tăng `quantity` của sách bằng cách sửa trực tiếp thông tin sách (không qua thao tác trả), người đang đứng trong hàng chờ sẽ không được tự động xét mượn, gây mất công bằng. Lỗi này đã được sửa triệt để bằng cách đảm bảo `serve_waiting_queue` được gọi ở mọi nơi `quantity` có thể tăng lên, không chỉ riêng luồng trả sách.

4 Báo cáo thống kê — `report.py`

Vị trí: `source_code/logic/report.py`

Mô-đun này không xử lý nghiệp vụ mới mà chỉ tổng hợp lại dữ liệu đã có để phục vụ hiển thị. `get_borrowing_loans` duyệt toàn bộ lịch sử giao dịch, lọc ra các phiếu có `status == "borrowing"` kèm tên sách tương ứng. `get_overdue_loans` làm tương tự nhưng lọc phiếu có `due_date` đã qua — đáng chú ý là hàm này tiện thể cập nhật luôn `status` của phiếu thành "overdue" ngay khi phát hiện, chứ không chỉ đọc dữ liệu. Cuối cùng, `get_top5_books` tận dụng lại `PriorityQueue` đã trình bày ở phần cấu trúc dữ liệu: đẩy toàn bộ sách có `borrow_count > 0` vào heap, rồi lấy ra 5 sách có lượt mượn cao nhất.

Hàm	**Độ phức tạp**
`get_borrowing_loans` / `get_overdue_loans`	$O(n)$
`get_top5_books`	$O(n \log n)$

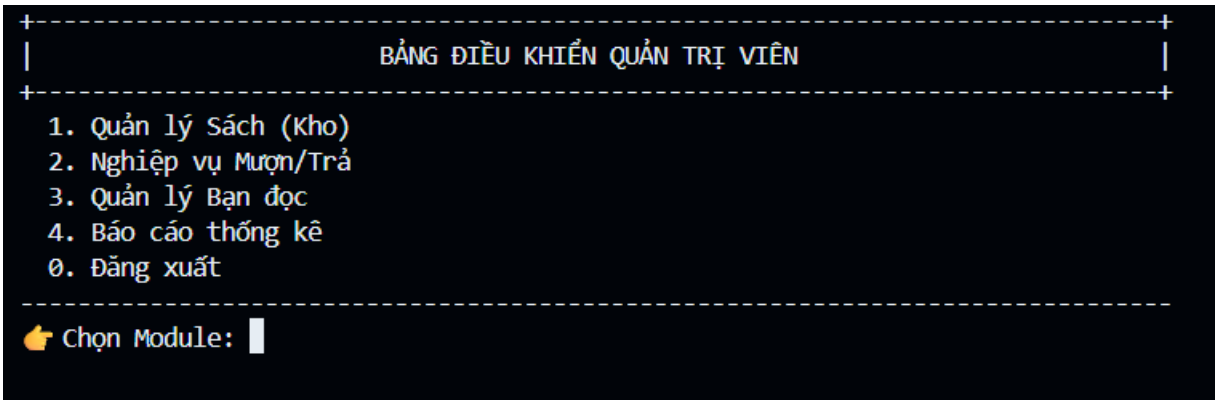
V Xây dựng giao diện

Hệ thống quản lý thư viện được thiết kế dưới dạng Giao diện dòng lệnh (CLI - Command Line Interface) nhằm tối ưu hóa hiệu suất bộ nhớ và tập trung trình diễn các cấu trúc dữ liệu. Giao diện được cấu trúc theo hệ thống Menu đa cấp (Multi-level Menu) với điều hướng rõ ràng, tích hợp các bộ lọc hiển thị bảng ASCII chuyên nghiệp.

1 Giao diện chính của chương trình

Giao diện chính của ứng dụng được xây dựng theo một cấu trúc có tổ chức, phân tách rõ ràng luồng truy cập của Quản trị viên (Admin) và Bạn đọc (Reader).

1.1 Cửa sổ chính



Điểm truy cập và Menu điều hướng: Sử dụng các vòng lặp vô hạn `while True` để duy trì màn hình ứng dụng, hệ thống điều hướng người dùng thông qua thao tác nhập phím số. Giao diện chính bao gồm phần Header (chứa tên hệ thống) và thân Menu liệt kê các Module chức năng.

1.2 Hệ thống tab chức năng

Để giao diện Console không bị rối mắt sau nhiều lần thao tác và xử lý khối lượng dữ liệu lớn mà không làm trôi màn hình, chương trình sử dụng các hàm tiện ích vẽ, làm sạch màn hình và phân trang:

- **Hàm `print_header`:**
  - **Input:** Chuỗi văn bản `title` (Tiêu đề của Menu hiện tại).
  - **Output:** Không có giá trị trả về. Thay đổi giao diện hiển thị trên Console.

– **Code sử dụng:**

```
def print_header(title: str):
```

– **Cơ chế hoạt động:** Trước tiên gọi lệnh xóa màn hình hệ thống (`os.system('cls'/'clear')`). Sử dụng phương thức `.center(78)` của string để tự động căn giữa tiêu đề và vẽ khung viền ASCII dạng `+--+` bao quanh, tạo diện mạo chuyên nghiệp.

– **Mục đích:** Đặt lại màn hình làm việc và thông báo cho người dùng biết họ đang ở phân hệ chức năng nào.

- **Hàm `_trigger_save`:**

  – **Input:** Tham chiếu đến các cấu trúc dữ liệu bộ nhớ: `hash_map`, `dll`, `user_array`, `waiting_queue`.

  – **Output:** Không hiển thị ra màn hình, xử lý ngầm (Background process).

  – **Code sử dụng:**

```
def _trigger_save(hash_map, dll, user_array, waiting_queue):
```

  – **Cơ chế hoạt động:** Xác định đường dẫn tương đối đến thư mục `data/` của dự án. Bọc trong khối `try/except` để gọi module `data_processor.save_system_data`, tiến hành ghi đè dữ liệu mới nhất từ RAM xuống các file CSV. Nếu có lỗi ghi file, in ra thông báo lỗi `[!]`.

  – **Mục đích:** Kích hoạt quá trình sao lưu dữ liệu tự động ngay sau bất kỳ thao tác nào làm thay đổi trạng thái của hệ thống (như Thêm sách, Mượn/Trả sách), đảm bảo tính Persistence (toàn vẹn dữ liệu).

- **Hàm `display_books_paginated` (Cơ chế in bảng ASCII và Phân trang):**

  – **Input:** `books` (Danh sách các đối tượng sách Book cần hiển thị).

  – **Output:** Các bảng ASCII trên Console, hiển thị từng trang danh sách sách.

  – **Code sử dụng:**

```
def display_books_paginated(books):
```

  – **Cơ chế hoạt động:** Kiểm tra rỗng. Nếu `books` rỗng, thông báo `[!]` Không có dữ liệu. Khởi tạo hằng số `PAGE_SIZE = 10` (10 dòng/trang) và khai báo độ rộng

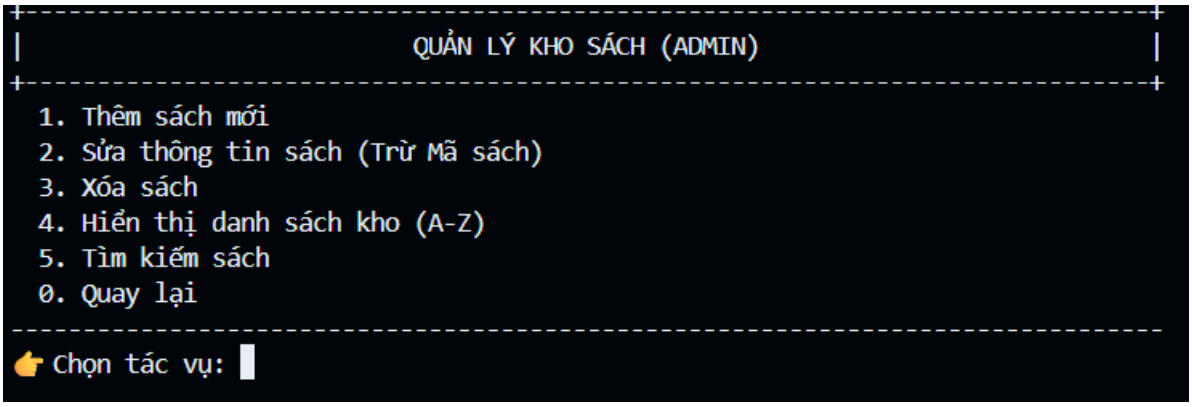
cố định cho từng cột. Dùng vòng lặp `for` với bước nhảy là `PAGE_SIZE` để cắt danh sách thành các chunk. Nếu tên sách dài hơn độ rộng cột cho phép, chuỗi sẽ bị cắt và nối thêm `...` ở cuối. Dùng hàm `.ljust()` và `.rjust()` để căn lề. Sau khi in hết 1 chunk, dùng hàm `input()` để tạo điểm dừng chờ người dùng nhấn Enter sang trang.

– **Mục đích:** Xây dựng trải nghiệm xem dữ liệu khoa học, giúp người dùng tra cứu thông tin sách dễ dàng mà không bị quá tải thông tin trên màn hình. Kỹ thuật tương tự cũng được áp dụng cho hàm `display_loans_paginated`.

2 Giao diện và thao tác với Sách

Lớp điều khiển UI quản lý các tương tác của Quản trị viên đối với danh mục Sách.

2.1 Giao diện người dùng



Sau khi truy cập Menu "QUẢN LÝ KHO SÁCH", giao diện in ra 5 lựa chọn: Thêm, Sửa, Xóa, Hiển thị (A-Z) và Tìm kiếm. Người dùng tương tác thông qua các câu lệnh nhắc nhập liệu tuần tự.

2.2 Thao tác người dùng

- **Hàm `admin_manage_books`:**
  - **Input:** Các cấu trúc dữ liệu lưu trữ: `hash_map`, `dll`, `user_array`, `waiting_queue`.
  - **Output:** Giao diện điều hướng và các thông báo kết quả cập nhật kho sách.
  - **Code sử dụng:**

```
def admin_manage_books(hash_map, dll, user_array, waiting_queue):
```

  - **Cơ chế hoạt động:**

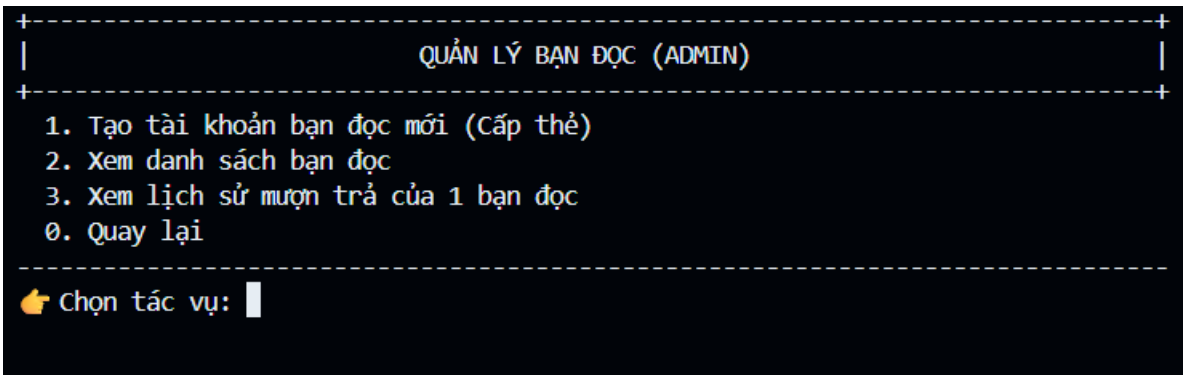
* *Nhập liệu và Kiểm duyệt:* Nhận `choice` từ người dùng. Với tác vụ Thêm (`choice == '1'`), giao diện lần lượt yêu cầu nhập các trường thông tin. Dữ liệu ngay lập tức được truyền qua module validator (ví dụ: `validate_quantity(qty_str)`). Nếu sai định dạng, vòng lặp ngắt sớm bằng `continue` để bắt nhập lại.
* *Cập nhật thông minh* (`choice == '2'`): Khi sửa thông tin, giao diện in ra giá trị hiện hành trong ngoặc vuông `[Giá trị cũ]`. Kỹ thuật `input() or book.title` cho phép người dùng ấn Enter để giữ nguyên giá trị cũ mà không cần nhập lại. Nếu số lượng `qty` mới lớn hơn `old_qty`, hàm tự động gọi `loan_manager.serve_waiting_queue` để cấp sách cho hàng đợi.
* *Khóa an toàn* (`choice == '3'`): Khi xóa, gọi hàm logic `is_book_on_loan` để kiểm tra. Nếu sách đang có người mượn, giao diện báo `[!]` Từ chối xóa.

– **Mục đích:** Quản lý vòng đời dữ liệu sách một cách chặt chẽ từ phía front-end (Console), không cho phép dữ liệu sai lệch lọt xuống tầng logic và lưu trữ.

3 Giao diện và thao tác với Bạn đọc

(Ghi chú: Lớp điều khiển UI quản lý các tương tác của Quản trị viên đối với danh sách Bạn đọc).

3.1 Giao diện người dùng



Cung cấp các lựa chọn liên quan đến quản lý độc giả, cấp thẻ mới và tra cứu thông tin cá nhân của từng người dùng trên hệ thống.

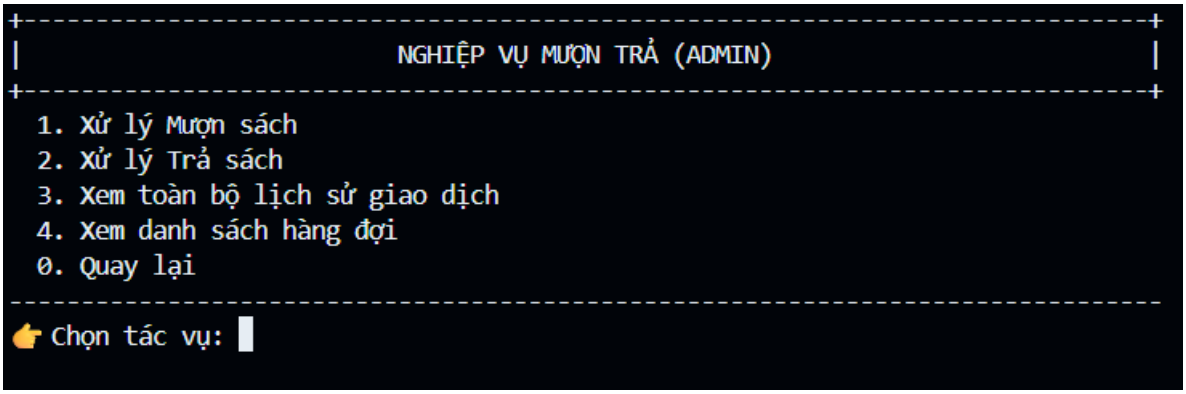
3.2 Thao tác người dùng

Người dùng nhập liệu tuần tự theo các nhắc lệnh trên màn hình để thêm mới tài khoản độc giả hoặc tra cứu mã sinh viên/giảng viên nhằm hiển thị lịch sử hoạt động.

4 Giao diện và thao tác với Mượn/Trả sách

Đây là phân hệ giao diện có logic phức tạp nhất, quản lý sự ràng buộc giữa Bạn đọc, Sách và Hàng chờ (Waiting Queue).

4.1 Giao diện người dùng



Chứa các lựa chọn Mượn sách, Trả sách, Xem lịch sử và Xem Hàng đợi. Khi nhập liệu, các cảnh báo tương tác thời gian thực sẽ xuất hiện nếu hệ thống phát hiện vấn đề (ví dụ: hết sách).

4.2 Thao tác người dùng

- **Hàm** `admin_manage_loans`**:**
  - **Input:** `hash_map`, `dll`, `user_array`, `waiting_queue`.
  - **Output:** Giao diện xử lý nghiệp vụ, thông báo mượn/trả và thu phí phạt.
  - **Code sử dụng:**

```
def admin_manage_loans(hash_map, dll, user_array, waiting_queue):
```

  - **Cơ chế hoạt động:**
    - *Nghiệp vụ Mượn* (`choice == '1'`): Sau khi nhập Mã độc giả và Mã sách, giao diện tra cứu sách trong `hash_map`. Điểm nhấn UX nằm ở Chốt chặn giao diện: Nếu `book.quantity <= 0`, một hộp thoại Console xuất hiện: `[CẢNH BÁO] Sách đã hết! Bạn có muốn vào hàng đợi không? (y/n)`. Tùy thuộc vào lựa chọn `y/n`, UI sẽ gọi hàm đưa người dùng vào `waiting_queue` hoặc thoát luồng, không gọi đến logic tạo phiếu mượn.

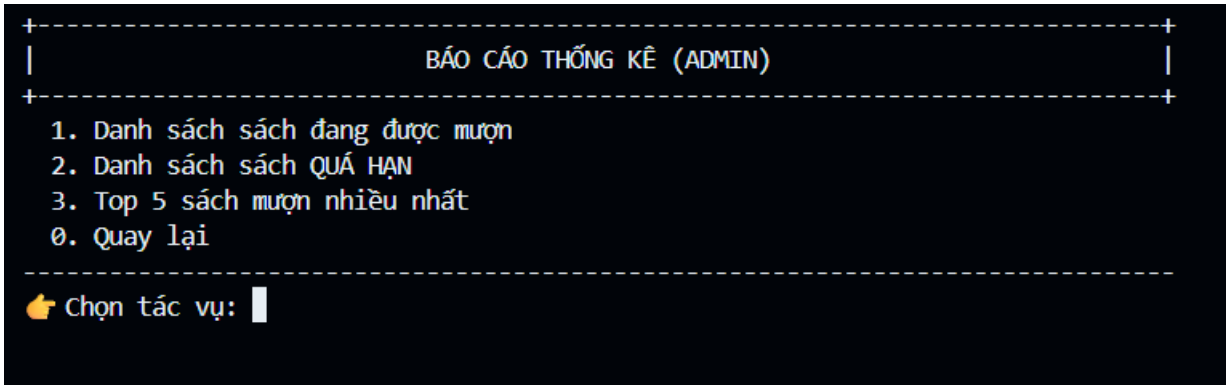
* *Nghiệp vụ Trả* (`choice == '2'`): Gọi hàm `process_return`. Nếu hàm logic trả về khoản phạt `fee > 0`, giao diện kích hoạt cảnh báo thu tiền: `[THU PHẠT] {fee} VNĐ (Trễ hạn)` để nhắc nhở thủ thư.
* *Thống kê lịch sử giao dịch* (`choice == '3'`): Cho phép truy xuất toàn bộ lịch sử giao dịch phát sinh trên hệ thống.
* *Tra cứu Hàng đợi* (`choice == '4'`): Gọi phương thức `waiting_queue.to_list()`. Giao diện bóc tách đối tượng `BorrowRequest` và in ra bảng danh sách chi tiết mã người dùng, mã sách và ngày đăng ký chờ, giúp quản trị viên theo dõi thực trạng cung cầu của thư viện.

– **Mục đích:** Xử lý triệt để các tình huống nghiệp vụ phức tạp ngay trên giao diện Console một cách trực quan, đảm bảo tính liên kết chặt chẽ giữa các sự kiện (Mượn - Hết sách - Chờ sách - Trả sách - Thu phạt).

5 Giao diện Báo cáo Thống kê (Admin)

Phân hệ này cung cấp các báo cáo tổng quan về tình hình hoạt động của thư viện, giúp Quản trị viên đưa ra các quyết định vận hành chính xác và theo dõi xu hướng mượn sách.

5.1 Giao diện người dùng



Khi truy cập Menu "BÁO CÁO THỐNG KÊ", giao diện hiển thị 3 tùy chọn báo cáo chính: Danh sách sách đang được mượn, Danh sách sách QUÁ HẠN, và Top 5 sách mượn nhiều nhất. Người dùng chọn số tương ứng để xuất báo cáo ra màn hình Console.

5.2 Thao tác người dùng

- **Hàm `admin_reports`:**
  – **Input:** Các cấu trúc dữ liệu lưu trữ: `hash_map`, `dll`, `user_array`.
  – **Output:** Các bảng danh sách thống kê trực quan trên màn hình.

– **Code sử dụng:**

```
def admin_reports(hash_map, dll, user_array):
  while True:
    choice = input("Chọn tác vụ: ").strip()
    if choice == '1': # Sách đang mượn
      data = report.get_borrowing_loans(dll, hash_map)
    elif choice == '2': # Sách quá hạn
      data = report.get_overdue_loans(dll, hash_map, user_array)
    elif choice == '3': # Top 5 sách
      pq = PriorityQueue()
      books = report.get_top5_books(hash_map, pq)
```

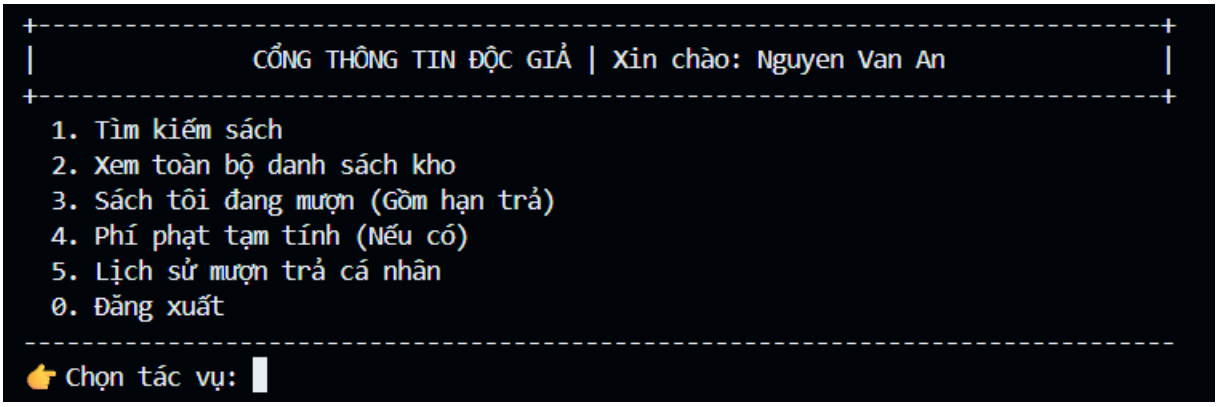
– **Cơ chế hoạt động:**

* *Sách đang mượn* (`choice == '1'`): Gọi hàm logic duyệt qua danh sách liên kết đôi (DLL) để lọc các phiếu có trạng thái `borrowing`.
* *Sách quá hạn* (`choice == '2'`): Lọc các phiếu có trạng thái `overdue`. Hàm cũng tự động tính toán lại khoản tiền phạt (`temp_fee`) tính đến ngày hiện hành để hiển thị.
* *Top 5 sách hot nhất* (`choice == '3'`): Tích hợp cấu trúc dữ liệu Hàng đợi ưu tiên (`PriorityQueue`). Hệ thống đẩy tất cả sách vào Queue dựa trên thuộc tính `borrow_count` (số lượt mượn) và rút ra 5 cuốn có lượt mượn cao nhất theo thứ tự giảm dần.

– **Mục đích:** Giúp quản trị viên kiểm soát được dòng chảy của sách, theo dõi sát sao nợ xấu (quá hạn) để thu hồi, và nắm bắt xu hướng đọc của sinh viên để có kế hoạch nhập thêm đầu sách phù hợp.

6 Giao diện Độc giả (User Portal)

Dành cho người sử dụng thông thường với quyền Read-Only (Chỉ đọc).



- **Hàm `run_user_menu`:**
  - **Input:** Các cấu trúc dữ liệu lưu trữ và `current_user` (Phiên đăng nhập của người dùng hiện tại).
  - **Output:** Giao diện cá nhân hóa của độc giả.
  - **Code sử dụng:**

```
def run_user_menu(hash_map, dll, user_array, waiting_queue,
current_user):
```

  - **Cơ chế hoạt động:**
    - *Cá nhân hóa:* Header của menu trích xuất `current_user.fullname` để hiển thị lời chào định danh.
    - *Hiển thị các chức năng:* Tìm kiếm sách, Xem toàn bộ sách trong kho, Lịch sử mượn trả cá nhân.
    - *Tra cứu Sách đang mượn* (`choice == '3'`): Lọc các giao dịch có status `borrowing` để hiển thị nhắc nhở người dùng về `due_date` (Hạn trả sách).
    - *Tra cứu Phí phạt* (`choice == '4'`): Giao diện quét toàn bộ lịch sử DLL, lọc ra các phiếu `overdue` thuộc về `user_id` hiện tại. Nó tự động gọi hàm `calculate_overdue_fee` (tính dựa trên `date.today()`) để cộng dồn và in ra màn hình Tổng tiền phạt tạm tính theo thời gian thực.
  - **Mục đích:** Cung cấp cổng thông tin minh bạch, bảo mật, giúp độc giả tự quản lý được thông tin tài khoản và khoản nợ của mình mà không cần can thiệp vào nghiệp vụ của thủ thư.

VI Kiểm thử

1 Phương pháp và công cụ kiểm thử

Phần này trình bày bộ test (93 test) áp dụng kỹ thuật lập trình phòng ngừa, chia thành hai nhóm: kiểm thử dữ liệu đầu vào (`test_validator.py`) và kiểm thử điều kiện nghiệp vụ (`test_sort.py`, `test_search.py`, `test_report.py`, `test_loan_manager.py`). Nhóm chọn `unittest` — module kiểm thử có sẵn trong Python, không cần cài thêm — làm công cụ thực hiện, vì nó cung cấp:

- Các phương thức `assert*` (`assertEqual`, `assertTrue`, `assertFalse`...) để so sánh kết quả thực tế với kết quả kỳ vọng, tự báo rõ khi sai.
- `setUp()` chạy lại trước mỗi test, đảm bảo mỗi test bắt đầu từ một bộ dữ liệu mẫu sạch, độc lập với các test khác.
- Cơ chế `discover` tự tìm và chạy mọi file `test_*.py`, không cần khai báo tay từng test.
- Bản tổng kết rõ ràng cuối cùng — số test đã chạy, thời gian chạy, và traceback chỉ rõ đúng dòng gây lỗi khi có test fail.

2 Cơ chế tổ chức và đánh giá một trường hợp kiểm thử

Khác với hàm thông thường, một test trong `unittest` không nhận tham số từ bên ngoài - dữ liệu đầu vào của nó là dữ liệu mẫu được viết cứng ngay trong thân hàm test, hoặc nạp sẵn từ CSV qua `tests/data_loader.py`. Để thấy rõ điều này, xét hai test sau:

```
def test_validate_quantity_valid_positive(self):
    self.assertTrue(validator.validate_quantity("5"))
```

Trong đoạn code trên, ‘‘-5’’ là dữ liệu đầu vào của test; `self.assertFalse(...)` so sánh kết quả gọi `validate_quantity(‘‘-5’’)` với giá trị kỳ vọng là `False`.

```
def test_borrow_out_of_stock(self):
    success, msg = loan_manager.process_borrow(
        self.hash_map, self.dll, self.user_array, "20211004", "B007", self.waiting_queue, borrow_date=date(2024, 1, 1)
    )
    self.assertFalse(success)
```

Tương tự, dữ liệu đầu vào ở đây là cặp `("20211004", "B007")` cùng dữ liệu mẫu đã nạp sẵn (`B007` có `quantity = 0`); kết quả của test là việc biến `success` lấy ra từ `process_borrow` có khớp với giá trị kỳ vọng `False` hay không.

Khi chạy `python -m unittest discover`, test thuộc tầng logic chỉ in ra dòng tổng kết `OK` vì tầng này không gọi `print()`. Trong khi đó, các test trong `test_validator.py` lại in kèm những dòng bắt đầu bằng `[!]` — không phải lỗi, mà vì các hàm `validate_*` vừa trả về `True`/`False`/`None`, vừa tự `print()` thông báo lỗi ngay khi phát hiện dữ liệu sai (như trong ví dụ `test_validate_quantity_negative` ở trên, hàm `validate_quantity` in dòng "Số lượng phải là một số nguyên dương không âm." trước khi trả `False`) — đúng vai trò của tầng giao diện là báo lỗi trực tiếp cho người dùng cuối, không phải lỗi của test.

3 Nguyên tắc lựa chọn dữ liệu kiểm thử

Việc chọn dữ liệu cho một test được tính toán để chọn ra các trường hợp biên, nhằm vào đúng những điểm dễ sai nhất khi viết điều kiện so sánh.

```
def test_validate_quantity_zero_is_valid(self):
    self.assertTrue(validator.validate_quantity("0"))
```

`test_validate_quantity_zero_is_valid` chọn input là `"0"`. Quy tắc hợp lệ trong code là `qty_str.isdigit() and int(qty_str) >= 0` — biên dưới của miền hợp lệ chính là `0`. Nhóm chọn đúng giá trị này vì nếu ai đó vô tình viết `> 0` thay vì `>= 0` (một lỗi lệch 1 đơn vị rất dễ gặp khi code điều kiện biên), test sẽ kỳ vọng `True` nhưng thực tế nhận được `False`, lập tức bắt được lỗi mà một test chọn số dương bất kỳ như `"5"` không thể phát hiện ra.

```
def test_validate_password_exactly_min_length(self):
    self.assertTrue(validator.validate_password("123456"))
```

`test_validate_password_exactly_min_length` chọn input là `"123456"` — đúng 6 ký tự. Quy tắc từ chối trong code là `len(password.strip()) < 6`. Nhóm chọn đúng giá trị bằng `6` để kiểm tra dấu so sánh `<` có bị viết nhầm thành `<=` không: nếu nhầm, một mật khẩu hợp lệ đúng 6 ký tự sẽ bị từ chối oan, và chỉ test ở đúng giá trị biên này mới lộ ra được lỗi đó.

```
def test_borrow_exceed_limit(self):
    success, msg = loan_manager.process_borrow(
        self.hash_map, self.dll, self.user_array, "20211003", "B001", self.waiting_queue, borrow_date=date(2024, 1, 1)
    )
    self.assertFalse(success)
```

`test_borrow_exceed_limit` chọn bạn đọc `20211003` — người đã mượn đúng bằng số lượng `borrow_limit` cho phép, không hơn không kém. Điều kiện từ chối trong `process_borrow` là `borrowing_count >= user.borrow_limit`. Nhóm chọn đúng trường hợp `borrowing_count == borrow_limit` vì đây là biên dễ viết sai nhất: nếu code lỡ viết `>` thay vì `>=`, một bạn

đọc đã mượn đủ giới hạn vẫn sẽ được mượn thêm một cuốn nữa, vượt quy định mà không bị test bắt được nếu chỉ thử với số lượng vượt hẳn giới hạn.

Các test còn lại trong bộ 93 test cũng được thiết kế theo cùng nguyên tắc — chọn đúng giá trị biên hoặc trường hợp đại diện cho từng nhóm điều kiện, không chỉ riêng bốn ví dụ nêu trên.

4 Kết quả kiểm thử

Chạy `python -m unittest discover` từ thư mục gốc dự án cho kết quả:

```
BKC@DESKTOP-E8NDGD0 MINGW64 /d/2025.2/KTLT/Báo cáo/G1-KTLT/KTLT-Library-Management-System (main)
$ python -m unittest discover
.....................................................................
[!] Lỗi: Mã sách phải bắt đầu bằng chữ 'B' kèm theo 3-4 chữ số (Ví dụ: B001, B102).
.
[!] Lỗi: Mã sách phải bắt đầu bằng chữ 'B' kèm theo 3-4 chữ số (Ví dụ: B001, B102).
...
[!] Lỗi: Trường dữ liệu 'name' không được bỏ trống.
.
[!] Lỗi: Trường dữ liệu 'name' không được bỏ trống.
..
[!] Lỗi: Mật khẩu quá ngắn. Cần tối thiểu 6 ký tự để đảm bảo an toàn.
.
[!] Lỗi: Mật khẩu quá ngắn. Cần tối thiểu 6 ký tự để đảm bảo an toàn.
..
[!] Lỗi: Tên/Tác giả chỉ được chứa chữ cái và khoảng trắng (2-50 ký tự).
.
[!] Lỗi: Tên/Tác giả chỉ được chứa chữ cái và khoảng trắng (2-50 ký tự).
.
[!] Lỗi: Tên/Tác giả chỉ được chứa chữ cái và khoảng trắng (2-50 ký tự).
..
[!] Lỗi: Số lượng phải là một số nguyên dương không âm.
.
[!] Lỗi: Số lượng phải là một số nguyên dương không âm.
....
[!] Lỗi: Ten sach quá dài (Tối đa 10 ký tự).
.
[!] Lỗi: Ten sach quá dài (Tối đa 50 ký tự).
..
[!] Lỗi: Định dạng mã độc giả không hợp lệ.
    - Sinh viên: Chỉ gồm 8 đến 9 chữ số (Ví dụ: 20231234).
    - Giảng viên: Phải theo định dạng 002.xxx.xxxxx (Ví dụ: 002.123.45678).
.....
[!] Lỗi: Định dạng mã độc giả không hợp lệ.
    - Sinh viên: Chỉ gồm 8 đến 9 chữ số (Ví dụ: 20231234).
    - Giảng viên: Phải theo định dạng 002.xxx.xxxxx (Ví dụ: 002.123.45678).
.
----------------------------------------------------------------------
Ran 93 tests in 0.146s

OK
```

VII Kĩ thuật lập trình áp dụng

Hệ thống quản lý thư viện được xây dựng không chỉ để giải quyết bài toán nghiệp vụ mà còn để trình diễn việc áp dụng linh hoạt, chính xác các kỹ thuật lập trình nền tảng. Dưới đây là các kỹ thuật tiêu biểu đã được triển khai trong dự án:

1 Kĩ thuật thiết kế Top-Down

Phương pháp thiết kế Top-Down yêu cầu "phân tích từ trên xuống, từ tổng quát tới chi tiết". Thay vì viết toàn bộ code vào một file, bài toán lớn được chia nhỏ thành các bài toán con, giúp việc kiểm soát logic trở nên dễ dàng.

- **Mức tổng quát nhất:** File `main.py` chỉ có nhiệm vụ khởi tạo các cấu trúc dữ liệu (Hash Map, DLL, Queue) và gọi các hàm Menu điều hướng cấp cao (`run_admin_menu` hoặc `run_user_menu`).
- **Mức trung gian:** Trong `interface/menu.py`, các menu cấp cao tiếp tục phân rã thành các phân hệ chức năng cụ thể: `admin_manage_books` (Quản lý Sách), `admin_manage_loans` (Quản lý Mượn/Trả).
- **Mức chi tiết nhất:** Tại các phân hệ, giao diện không tự xử lý dữ liệu mà gọi đến các hàm nghiệp vụ ở tầng Logic như `loan_manager.process_borrow()` hay `search.search_by_title()`.

2 Kĩ thuật module hoá

Dự án tuân thủ nghiêm ngặt nguyên lý "Tách biệt mối quan tâm" (Separation of Concerns). Mã nguồn được chia thành các thư mục (package) và file (module) với trách nhiệm duy nhất (Single Responsibility Principle):

- `objects/`: Định nghĩa các kiểu dữ liệu có cấu trúc (`Book`, `User`, `Loan`).
- `structure/`: Tách biệt phần tự cài đặt Cấu trúc dữ liệu (`hash_map.py`, `doubly_linked_list.py`).
- `logic/`: Chứa các quy tắc nghiệp vụ (Business rules). Ví dụ: `loan_manager.py` chỉ lo việc kiểm tra điều kiện mượn/trả, hoàn toàn không chứa lệnh `print()` hay `input()`.
- `interface/`: Chịu trách nhiệm giao tiếp với người dùng và bắt lỗi nhập liệu.
- `storage/`: Chuyên biệt cho việc đọc/ghi file CSV.

Ưu điểm: Nhờ sự phân tách này, nếu sau này cần chuyển đổi từ giao diện Console sang giao diện Web hoặc thay đổi cách lưu file CSV sang Database (SQL), ta chỉ cần sửa đúng module tương ứng mà không làm "vỡ" toàn bộ hệ thống.

3 Cấu trúc dữ liệu và giải thuật

Dự án thay thế việc sử dụng các kiểu dữ liệu mặc định bằng việc tự triển khai (Implement) các cấu trúc dữ liệu chuyên sâu để tối ưu hóa bài toán thực tế.

3.1 Lựa chọn và triển khai cấu trúc dữ liệu

- **Bảng băm (Hash Map):** Được sử dụng làm cấu trúc lưu trữ Kho sách. Nhờ cơ chế băm khóa (Hashing) từ `book_id`, thao tác tra cứu, cập nhật thông tin sách đạt độ phức tạp thời gian trung bình là $O(1)$, vượt trội so với tìm kiếm tuần tự trên mảng.
- **Danh sách liên kết đôi (Doubly Linked List):** Áp dụng để lưu trữ Lịch sử giao dịch (Loans). Mỗi nút lưu cả con trỏ `prev` và `next`, cho phép mở rộng duyệt hai chiều khi cần trong tương lai; hiện tại hệ thống khai thác chiều thuận (`next`) để liệt kê lịch sử đúng theo trình tự thời gian, và tận dụng con trỏ `tail` để thêm giao dịch mới vào cuối danh sách với chi phí $O(1)$ mà không cần dịch chuyển hay cấp phát lại các phần tử đã có.
- **Hàng đợi (Waiting Queue):** Ứng dụng mô hình FIFO (First-In, First-Out) để giải quyết nghiệp vụ "Hết sách". Độc giả đăng ký chờ được đẩy vào cuối hàng đợi (`enqueue`). Khi sách được trả, hệ thống duyệt hàng đợi theo đúng thứ tự đăng ký để tìm người chờ đúng cuốn sách đó, rút người này ra bằng `remove_match()` nếu họ còn đủ điều kiện mượn (chưa nợ quá hạn, chưa vượt hạn mức); nếu không đủ điều kiện, người đó vẫn được giữ chỗ trong hàng đợi để xét lại ở lượt trả sách tiếp theo, đồng thời hệ thống chuyển sang xét người kế tiếp khớp cùng cuốn sách.

3.2 Giải thuật áp dụng

- **Giải thuật QuickSort:** Triển khai để sắp xếp danh mục sách theo Tên tác giả hoặc Tên sách (từ A-Z) với độ phức tạp thời gian trung bình $O(N \log N)$ trước khi đẩy dữ liệu lên hệ thống phân trang của giao diện.

4 Xử lý lỗi và ghi log

4.1 Xử lý lỗi

Thay vì sử dụng hệ quản trị cơ sở dữ liệu lớn, dự án lưu trữ qua file vật lý (CSV). Toàn bộ khối lệnh thao tác Đọc/Ghi file được đặt trong cấu trúc `try...except Exception as e`. Nếu quá trình thao tác File I/O gặp lỗi (ví dụ file đang bị phần mềm khác mở khóa/chỉ đọc), chương trình sẽ bắt lỗi và ngăn không cho ứng dụng bị crash (văng ra màn hình desktop).

4.2 Ghi log

Hệ thống áp dụng hai cơ chế ghi nhận song song, tuỳ theo loại lỗi. Đối với lỗi phát sinh khi đọc dữ liệu từ file CSV (ví dụ một dòng dữ liệu bị thiếu cột hoặc sai kiểu), hàm nội bộ `_log_error()` trong `storage/data_processor.py` sẽ ghi chi tiết dòng lỗi (kèm thời gian, tên file, nội dung dòng) vào file `data/system_error.log`, đồng thời bỏ qua dòng đó để không làm gián đoạn việc nạp các dữ liệu hợp lệ còn lại. Đối với lỗi vận hành tức thời (ví dụ lưu file thất bại), hệ thống xuất cảnh báo trực tiếp ra màn hình Console, ví dụ `[!] Lỗi khi tự động lưu dữ liệu: {e}`, giúp người quản trị nắm bắt sự cố ngay tại thời điểm phát sinh.

5 Đặt tên rõ ràng và nhất quán

Dự án tuân thủ nghiêm ngặt các quy tắc viết mã sạch (Clean Code) và chuẩn PEP 8 của Python:

- **Quy tắc đặt tên:** Tên Lớp (Class) sử dụng PascalCase (`Book`, `BorrowRequest`). Tên hàm và biến sử dụng snake_case mang ý nghĩa động từ/danh từ tự giải thích (`process_borrow`, `display_books_paginated`, `waiting_queue`). Không sử dụng các biến viết tắt vô nghĩa như `a`, `b`, `c`.
- **Mã hóa bằng Hằng số (Constants):** Các giá trị cố định như phí phạt (`OVERDUE_FEE = {"student": 2000, "lecturer": 1000}`) được tách ra thành hằng số ở đầu file. Điều này giúp việc bảo trì và thay đổi quy định thư viện sau này diễn ra nhanh chóng mà không cần rà soát sửa lỗi từng dòng code (Hard-code).

6 Chú thích

Để hỗ trợ làm việc nhóm và bảo trì sau này, toàn bộ các hàm cốt lõi đều được tài liệu hóa kỹ lưỡng:

- **Docstrings:** Đặt ngay bên dưới định nghĩa hàm để giải thích tham số và kết quả trả về (ví dụ: `"""Xử lý trả sách - tìm phiếu gần nhất, tính phạt, cập nhật, ưu tiên hàng chờ. Trả về (bool, str, float)."""`).
- **Comments:** Trong các khối code có logic phức tạp (như ưu tiên duyệt hàng đợi ở hàm Trả sách), comment dòng (`#`) được sử dụng để giải thích *tại sao* đoạn code đó tồn tại, giúp luồng thuật toán trở nên tường minh với người đọc mã nguồn.

7 Lập trình phòng ngừa

Một hệ thống tốt không được phép sụp đổ (crash) vì người dùng nhập sai, và không được sinh ra "dữ liệu rác". Kỹ thuật này được áp dụng qua 3 lớp bảo vệ:

7.1 Kiểm tra tính hợp lệ của Dữ liệu đầu vào (Input Validation)

Tại tầng giao diện (`menu.py`), mọi dữ liệu thô lập tức đi qua module `validator.py`. Hệ thống ép kiểu độ dài chuỗi, kiểm tra bắt buộc nhập (không được để trống `""`), và dùng `.isdigit()` kết hợp kiểm tra giá trị không âm để đảm bảo Số lượng sách luôn là số nguyên không âm (cho phép giá trị 0 khi sách tạm hết hàng nhưng vẫn còn lưu hành).

7.2 Kiểm tra các Điều kiện Nghiệp vụ

Ngay cả khi dữ liệu đầu vào đúng định dạng, hệ thống vẫn kiểm tra logic thực tế. Ví dụ, trong hàm `process_borrow()`, hệ thống chặn luồng và trả về `False` nếu: Kho sách `quantity <= 0`, Độc giả đang nợ sách quá hạn, hoặc độc giả đang mượn chính cuốn sách đó chưa trả.

7.3 Đảm bảo Tính Ổn định và Toàn vẹn Dữ liệu

- **Bảo vệ toàn vẹn tham chiếu (Referential Integrity):** Trước khi cho phép Admin Xóa một cuốn sách, hệ thống gọi hàm `is_book_on_loan()` quét qua DLL. Nếu sách đang có người mượn, thao tác xóa bị chặn đứng để tránh việc tạo ra các "phiếu mượn mồ côi" (lỗi mất tham chiếu dữ liệu).
- **Auto-save (Ghi tự động):** Hàm `_trigger_save()` được gắn vào sau mọi thao tác làm thay đổi trạng thái (Thêm, Sửa, Xóa, Mượn, Trả), đảm bảo dữ liệu luôn được đồng bộ xuống ổ cứng.

8 Đánh giá tổng kết

8.1 Điểm mạnh

- **Ứng dụng cấu trúc dữ liệu linh hoạt:** Việc tự xây dựng các cấu trúc dữ liệu từ con số 0 (như Hash Map, DLL, Queue) thay vì dùng các thư viện có sẵn thể hiện sự am hiểu sâu sắc về nền tảng kỹ thuật lập trình.
- **Kiến trúc mở rộng tốt:** Hệ thống áp dụng triệt để nguyên lý thiết kế phân tầng. Sự độc lập giữa giao diện Console và xử lý Business Logic là tiền đề cực kỳ lý tưởng nếu muốn nâng cấp phần mềm lên giao diện GUI hoặc Web App sau này.
- **Luồng nghiệp vụ chặt chẽ:** Gần như mọi thao tác có nguy cơ gây lỗi, từ sai sót nhập liệu đến các khe hở logic (như mượn sách khi kho bằng 0) đều được rào chắn toàn diện.

8.2 Điểm yếu

- **Giao diện dòng lệnh (CLI):** Dù giúp chương trình chạy rất mượt và ít tốn RAM, giao diện Console chưa đem lại trải nghiệm thân thiện và thao tác trỏ chuột nhanh chóng như giao diện đồ họa (GUI).
- **Hạn chế của thao tác File CSV:** Việc lưu trữ thông qua file vật lý CSV chỉ tối ưu với quy mô thư viện vừa và nhỏ. Khi khối lượng giao dịch lên tới hàng trăm ngàn bản ghi, quá trình đọc/ghi toàn bộ file (Overwriting) sau mỗi thao tác có thể gây ra hiện tượng thắt nút cổ chai (Bottleneck).

VIII Kết luận

Qua dự án xây dựng hệ thống quản lý thư viện, nhóm chúng em đã học được cách thiết kế và triển khai một chương trình quản lý hoàn chỉnh từ những bước đầu tiên. Bắt đầu từ việc xác định các đối tượng cốt lõi - Sách (Book), Bạn đọc (User), Phiếu mượn/trả (Loan) và Yêu cầu chờ mượn (BorrowRequest) — mỗi đối tượng đảm nhiệm việc lưu trữ dữ liệu và chuyển đổi qua lại với từng dòng bản ghi để thuận tiện cho việc lưu trữ trên file CSV, nhóm đã tự tay cài đặt các cấu trúc dữ liệu nền tảng thay vì sử dụng thư viện có sẵn: một mảng động tự quản lý capacity (`CustomList`) làm nền chung cho mọi cấu trúc khác, bảng băm (`BookHashMap`) giúp tra cứu sách và xử lý va chạm hiệu quả, danh sách liên kết kép (`TransactionList`) để quản lý lịch sử giao dịch mượn/trả theo trình tự thời gian, hàng đợi ưu tiên dạng Max-Heap (`PriorityQueue`) phục vụ thống kê các đầu sách được mượn nhiều nhất, và hàng đợi FIFO (`WaitingQueue`) để xử lý danh sách độc giả chờ mượn khi sách hết kho. Bên cạnh đó, nhóm cũng xây dựng cơ chế đọc - ghi dữ liệu nhằm nạp và lưu thông tin liên tục giữa các lần chạy chương trình thông qua các file CSV.

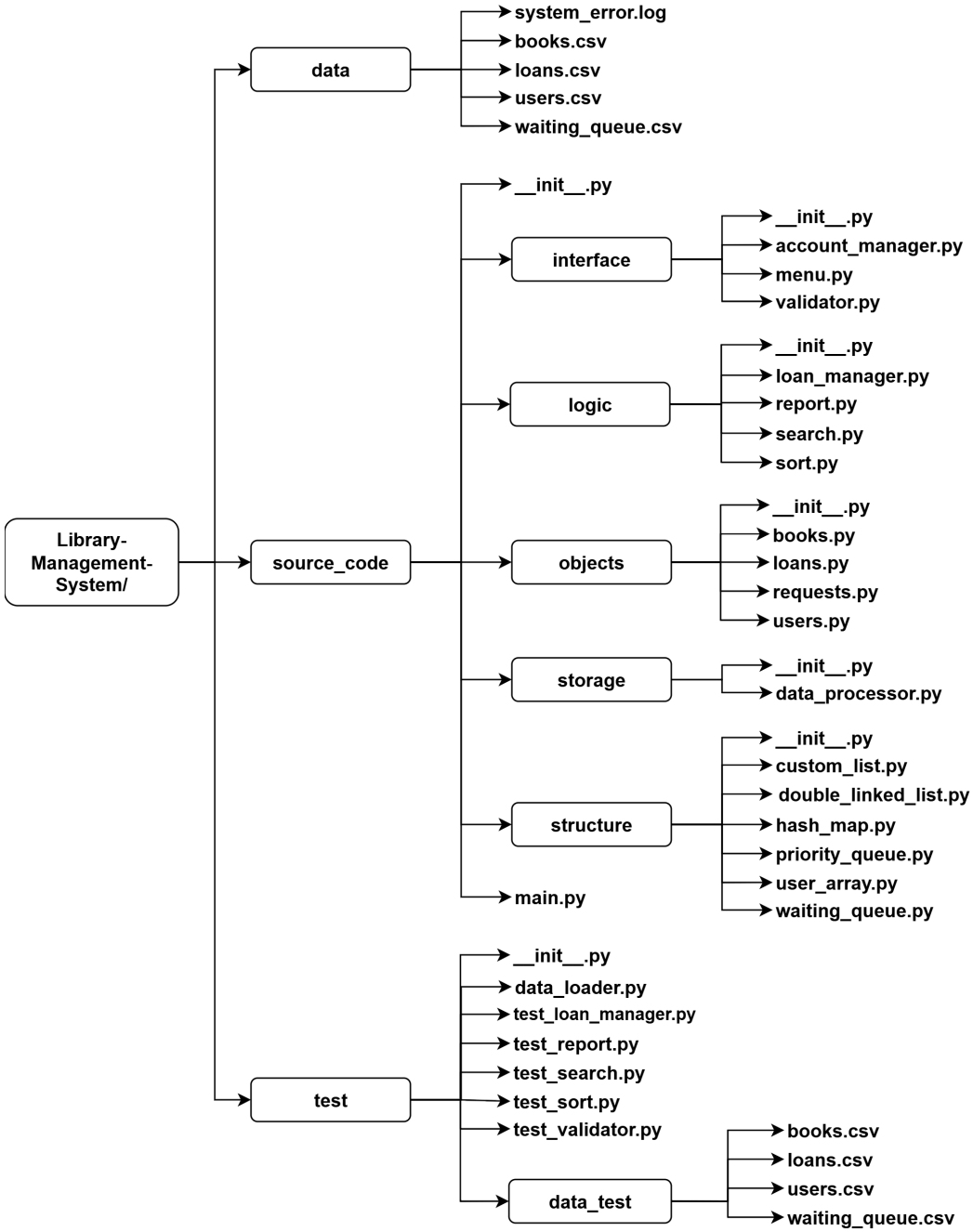
Trong quá trình thực hiện, nhóm đã rèn luyện được nguyên tắc tách biệt mối quan tâm (Separation of Concerns) khi chia mã nguồn thành năm tầng độc lập - thực thể đối tượng, cấu trúc dữ liệu, logic nghiệp vụ, giao diện và lưu trữ — mỗi tầng chỉ chịu trách nhiệm đúng một nhiệm vụ và giao tiếp với tầng khác qua các hàm rõ ràng, giúp việc kiểm tra tính hợp lệ dữ liệu đầu vào, xử lý nghiệp vụ mượn/trả, lưu trữ xuống file và cập nhật giao diện làm việc có thể phát triển và sửa lỗi độc lập với nhau. Từ góc độ môn học, nhóm đã hiểu rõ hơn về lập trình hướng đối tượng, cách tự cài đặt cấu trúc dữ liệu và giải thuật, kỹ thuật vào - ra với file CSV, quản lý ngoại lệ, cũng như các quy tắc tốt về kiểm tra dữ liệu và tổ chức mã nguồn. Nhờ đó, nhóm tự tin hơn trong việc phát triển những ứng dụng quy mô nhỏ và có thể mở rộng sang các dự án phức tạp hơn trong tương lai.

Lời cuối, nhóm chúng em xin chân thành cảm ơn thầy Vũ Thành Nam đã tận tình giảng dạy và chia sẻ kiến thức, giúp chúng em hoàn thiện hệ thống quản lý thư viện này.

IX Mã nguồn

1 Cấu trúc mã nguồn

Toàn bộ mã nguồn và dữ liệu của hệ thống được tổ chức theo cây thư mục sau:



Năm tầng `storage/`, `objects/`, `structure/`, `logic/`, `interface/` tương ứng với nguyên lý phân tách trách nhiệm (Separation of Concerns) đã trình bày ở mục Kĩ thuật module hoá. Riêng `tests/` chứa bộ dữ liệu mẫu (`data_test/`) và các bộ kiểm thử tự động hoàn toàn tách biệt với dữ liệu vận hành thật trong `data/`.

2 Mô tả file dữ liệu

Hệ thống lưu trữ bốn loại dữ liệu nghiệp vụ dưới dạng file CSV trong thư mục `data/`, cùng một file log lỗi.

books.csv

Cột	Ý nghĩa
book_id	Mã sách (ví dụ: B001)
title	Tên sách
author	Tác giả của cuốn sách
genre	Thể loại sách
publisher	Nhà xuất bản
quantity	Số lượng tồn kho
status	Trạng thái hiện tại của sách (active/inactive)
borrow_count	Số lượt mượn tích lũy, phục vụ thống kê Top 5

Dữ liệu mẫu được nhóm xây dựng có chủ đích để phủ đủ các nhánh xử lý: có sách `quantity = 0` (kiểm thử nghiệp vụ hết kho và hàng chờ), có sách `status = "inactive"` (kiểm thử từ chối mượn do ngừng lưu hành), và một số sách có `borrow_count` cao để minh họa báo cáo Top 5.

users.csv

Cột	Ý nghĩa
user_id	Mã định danh - xem quy ước riêng bên dưới
fullname	Tên người dùng
password	Mật khẩu
role	admin / user
reader_type	student / lecturer / rỗng (admin)

Quy ước `user_id` theo đúng định dạng thực tế của trường:

- **Sinh viên:** dùng đúng Mã số sinh viên (MSSV), gồm 8 hoặc 9 chữ số liên tiếp - ví dụ `202419090` (sinh viên nhập học năm 2024). Quy tắc này được `validator.validate_user_id()` kiểm tra bằng regex `▮\d{8,9}$`.

- **Giảng viên:** dùng đúng mã cán bộ theo cấu trúc `A.B.C`, trong đó `A` gồm 3 chữ số là mã phân loại chung dành cho Cán bộ Giảng viên cơ hữu của Đại học, `B` là mã định danh của Khoa/Viện hoặc Trung tâm chuyên môn mà giảng viên đó trực thuộc (3 chữ số), `C` là số thứ tự định danh cá nhân (5 chữ số) (ví dụ: `002.010.00042`). Được kiểm tra bằng regex `▮002\.\d{3}\.\d{5}$`.
- **Quản trị viên (Admin):** dùng định danh cố định `"admin"`, không theo hai quy tắc trên vì không phải đối tượng bạn đọc, không áp dụng hạn mức mượn (`reader_type` để rỗng).

Việc dùng đúng định dạng MSSV/mã cán bộ thực tế (thay vì mã giả định tùy ý) giúp logic phân loại tự động `reader_type` trong `account_manager.create_reader()` phản ánh đúng tình huống vận hành của một thư viện trường đại học.

loans.csv

Cột	**Ý nghĩa**
loan_id	Mã phiếu mượn
user_id, book_id	Khóa ngoại liên kết tới User và Book
borrow_date, due_date, return_date	Ba mốc thời gian của một phiếu mượn
status	borrowing / returned / overdue
overdue_fee	Tiền phạt (VNĐ),

Dữ liệu mẫu cố ý bao gồm đủ ba trạng thái (`borrowing`, `overdue`, `returned`) - trong đó có cả phiếu `returned` với `overdue_fee > 0` (trả trễ) và `= 0` (trả đúng hạn), để minh họa rõ công thức tính phạt khi kiểm thử.

waiting_requests.csv

Cột	**Ý nghĩa**
request_id	Mã yêu cầu chờ
user_id, book_id	Bạn đọc đang chờ, sách đang chờ
request_date	Ngày đăng ký chờ

File này khởi đầu chỉ chứa dòng header, vì hàng chờ chỉ phát sinh khi có người đăng ký mượn một cuốn sách đang hết kho trong quá trình sử dụng thực tế. Nhóm đã thêm trước 1 request để minh hoạ.

system_error.log

Không phải dữ liệu nghiệp vụ, mà là log lỗi tự động của `storage/data_processor.py` - ghi lại các dòng dữ liệu CSV không hợp lệ (sai kiểu, thiếu cột) gặp phải khi nạp hệ thống, kèm thời gian phát sinh, để admin tra cứu khi cần mà không làm gián đoạn quá trình nạp các dòng hợp lệ còn lại.

3 Chi tiết file main.py

`main.py` là điểm khởi chạy duy nhất của toàn hệ thống: không tự xử lý nghiệp vụ, chỉ điều phối việc nạp dữ liệu, đăng nhập, phân luồng vào menu và lưu dữ liệu.

```python
"""
Điểm khởi chạy hệ thống (Entry Point)
Nhiệm vụ: Khởi động chương trình, nạp dữ liệu từ CSV lên RAM, điều hướng đến menu Admin hoặc User.
"""

import os
import sys

# Import các package theo đúng cấu trúc thư mục
from storage import data_processor
from interface import account_manager, menu

def get_data_paths():
    """Hàm phụ trợ: Định vị hoặc tạo thư mục chứa file CSV an toàn trên mọi hệ điều hành."""
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    data_dir = os.path.join(base_dir, "data")

    if not os.path.exists(data_dir):
        os.makedirs(data_dir)

    books_path = os.path.join(data_dir, "books.csv")
    users_path = os.path.join(data_dir, "users.csv")
    loans_path = os.path.join(data_dir, "loans.csv")
    waiting_path = os.path.join(data_dir, "waiting_requests.csv")

    return books_path, users_path, loans_path, waiting_path

def main():
    # Nhận đủ 4 đường dẫn file
    books_path, users_path, loans_path, waiting_path = get_data_paths()

    print("\n[Hệ thống] Đang khởi động và kết nối cơ sở dữ liệu...")
    try:
        # Bước 1: Nạp đủ 4 cấu trúc dữ liệu (Thêm waiting_queue)
        hash_map, dll, user_array, waiting_queue = data_processor.load_system_data(
            books_path, users_path, loans_path, waiting_path
        )
    except Exception as e:
        print(f"[!] Lỗi nghiêm trọng khi nạp hệ thống: {e}")
        sys.exit(1)

    # Vòng lặp duy trì phần mềm
```

```
    # Vòng lặp duy trì phần mềm
    while True:
        # Làm sạch màn hình trước khi hiện form đăng nhập
        os.system('cls' if os.name == 'nt' else 'clear')
        print("+" + "-" * 78 + "+")
        print(f"|{'HỆ THỐNG QUẢN LÝ THƯ VIỆN ĐẠI HỌC'.center(78)}|")
        print("+" + "-" * 78 + "+")
        print(" (Gõ 'exit' hoặc '0' ở ô Tài khoản để tắt phần mềm)\n")

        # Bước 2: Hiển thị màn hình đăng nhập
        current_user = account_manager.login(user_array)

        # Xử lý thoát chương trình
        if current_user == "EXIT_SIGNAL":
            print("\n[Hệ thống] Đang lưu dữ liệu toàn cục...")
            try:
                # Truyền đủ 4 object và 4 đường dẫn để lưu file
                data_processor.save_system_data(
                    hash_map, dll, user_array, waiting_queue,
                    books_path, users_path, loans_path, waiting_path
                )
                print("[Hệ thống] Đã đồng bộ an toàn. Tạm biệt!")
            except Exception as e:
                print(f"[!] Lỗi khi ghi file CSV: {e}")
            break

        # Nếu đăng nhập sai thì lặp lại
        if current_user is None:
            input("\n👉 Nhấn Enter để thử lại...")
            continue

        # Bước 3: Phân luồng điều hướng (Nhớ truyền thêm waiting_queue)
        if current_user.is_admin():
            menu.run_admin_menu(hash_map, dll, user_array, waiting_queue)
        else:
            menu.run_user_menu(hash_map, dll, user_array, waiting_queue, current_user)

        data_processor.save_system_data(
            hash_map, dll, user_array, waiting_queue,
            books_path, users_path, loans_path, waiting_path
        )
```

```

if __name__ == "__main__":
    # Đảm bảo có thể chạy module bằng lệnh python source_code/main.py
    sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
    main()

```

Hàm phụ trợ get_data_paths()

Xác định đường dẫn tuyệt đối tới bốn file CSV trong `data/`, dựa trên vị trí thực tế của `main.py` trên đĩa (qua `os.path.abspath(__file__)`) — nhờ vậy chương trình chạy đúng

dù được gọi từ thư mục nào, không phụ thuộc vào đường dẫn tương đối có thể sai lệch theo nơi gõ lệnh. Nếu thư mục `data/` chưa tồn tại, hàm tự tạo trước khi trả về 4 đường dẫn.

Luồng chạy chính trong hàm main()

1. **Nạp dữ liệu:** Gọi `data_processor.load_system_data()` một lần duy nhất khi khởi động, nạp toàn bộ books/users/loans/waiting_requests từ CSV lên bốn cấu trúc dữ liệu trên RAM (`BookHashMap`, `TransactionList`, `UserArray`, `WaitingQueue`). Nếu bước này gặp lỗi nghiêm trọng, chương trình in cảnh báo và thoát ngay (`sys.exit(1)`) thay vì chạy tiếp với dữ liệu không đáng tin.

2. **Vòng lặp đăng nhập:** Lặp vô hạn (`while True`), mỗi lượt xóa màn hình, in tiêu đề hệ thống, rồi gọi `account_manager.login()`.

3. **Ba nhánh kết quả đăng nhập:**

   - Nếu người dùng gõ `"0"` hoặc `"exit"` (trả về chuỗi đặc biệt `"EXIT_SIGNAL"`): gọi `save_system_data()` lưu toàn bộ trạng thái RAM xuống CSV, in lời chào tạm biệt, rồi `break` thoát vòng lặp — kết thúc chương trình.
   - Nếu đăng nhập sai (`None`): yêu cầu nhấn Enter rồi `continue` quay lại đầu vòng lặp, không xử lý gì thêm.
   - Nếu đăng nhập đúng: dùng `current_user.is_admin()` để quyết định gọi `menu.run_admin_menu()` hoặc `menu.run_user_menu()`.

4. **Lưu sau mỗi phiên:** Ngay sau khi người dùng đăng xuất khỏi menu (hàm `run_admin_menu`/`run_user_menu` trả về), `main()` gọi lại `save_system_data()` một lần nữa, độc lập với việc `menu.py` cũng tự lưu sau mỗi thao tác thay đổi dữ liệu (`_trigger_save()`) — đây là lớp bảo vệ thứ hai, đảm bảo dữ liệu luôn được đồng bộ xuống đĩa ngay khi một phiên làm việc kết thúc, không phụ thuộc hoàn toàn vào việc mọi nơi trong menu đều gọi lưu đúng.

Khối `if __name__ == "__main__"`

Trước khi gọi `main()`, chương trình thêm thư mục gốc của dự án vào `sys.path` (`sys.path.append(...)`). Việc này cho phép chạy trực tiếp `python source_code/main.py` từ bất kỳ thư mục nào mà các lệnh `import` kiểu `from objects.books import Book` ở những module khác vẫn hoạt động đúng, vì Python tìm thấy package `objects`, `logic`... ngay trong đường dẫn vừa được thêm vào.

4 Phụ lục link code

Toàn bộ mã nguồn của dự án được lưu trữ tại GitHub:

https://github.com/thuyphuong180206-star/KTLT-Library-Management-System

PHÂN CÔNG CÔNG VIỆC

Họ và tên	**Nhiệm vụ**	**Đánh giá mức độ hoàn thành**	**Ký tên**
Hoàng Thủy Phương	- Thiết kế kiến trúc hệ thống - Code Structure, Logic - Kiểm thử, sửa lỗi nghiệp vụ và logic giữa các module - Nội dung báo cáo: Cấu trúc dữ liệu, Logic nghiệp vụ, Kiểm thử	10	
Cao Đỗ Hoàng Sơn	- Đặc tả module hệ thống - Tạo data mẫu - Code Storage, Object - Sửa lỗi nghiệp vụ và logic giữa các module - Nội dung báo cáo: Cấu trúc đối tượng, Kết luận, Mã nguồn	10	
Hoàng Minh Sơn	- Code main, interface - Nội dung báo cáo: Mô tả thiết kế, Xây dựng giao diện, Kỹ thuật lập trình áp dụng - Sửa lỗi nghiệp vụ	10	

TÀI LIỆU THAM KHẢO

1. **Đỗ Xuân Lôi (2006)**, "Cấu trúc dữ liệu và giải thuật", Nhà xuất bản ĐHQG Hà Nội.

2. **Nguyễn Đức Nghĩa (2007)**, "Cấu trúc dữ liệu và thuật toán", Nhà xuất bản Đại học Bách Khoa Hà Nội.

3. Slide bài giảng học phần MI3310 – Kỹ thuật lập trình của TS. Vũ Thành Nam.